

وزارت علوم تحقیقات فناوری

دانشگاه فنی و حرفه ای - دانشکده شمسی پور تهران

طراحی الگوریتم تدوین : دکتر عادلۃ نیا

وزارت علوم تحقیقات فناوری



دانشگاه فنی و حرفه ای

دانشکده شهید شمسی پور تهران

درس طراحی الگوریتم

جلسه هفتم

مقطع کارشناسی (نرم افزار)

مدرس : دکتر عادلۃ نیا

الگوریتم به سری فیبوناچی 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

$$\begin{cases} F(n) = F(n-1) + F(n-2) \\ F(0) = 0 \\ F(1) = 1 \end{cases}$$

حاصل جمع دو عدد قبلی برابر با عدد بعدی

روش: تکیه بر تقسیم و فتح
Divide and conquer

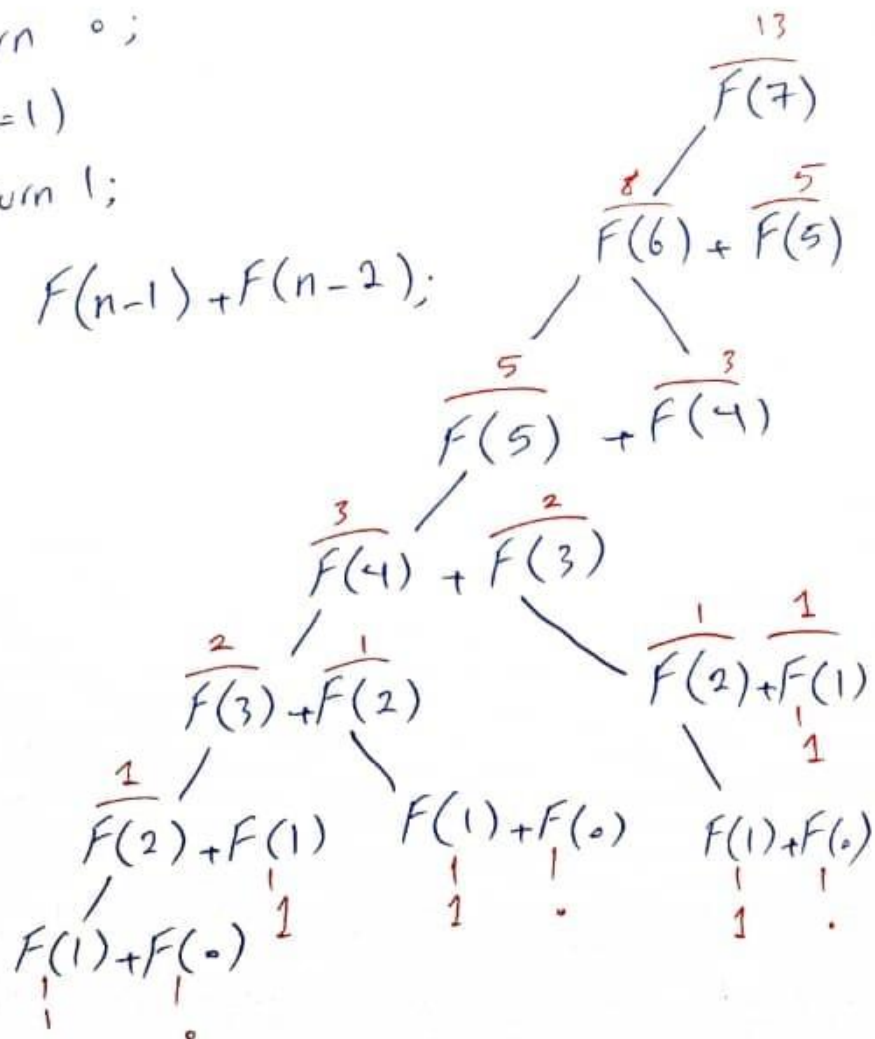
```
int F(int n)
{
```

```
    if (n == 0)
        return 0;
```

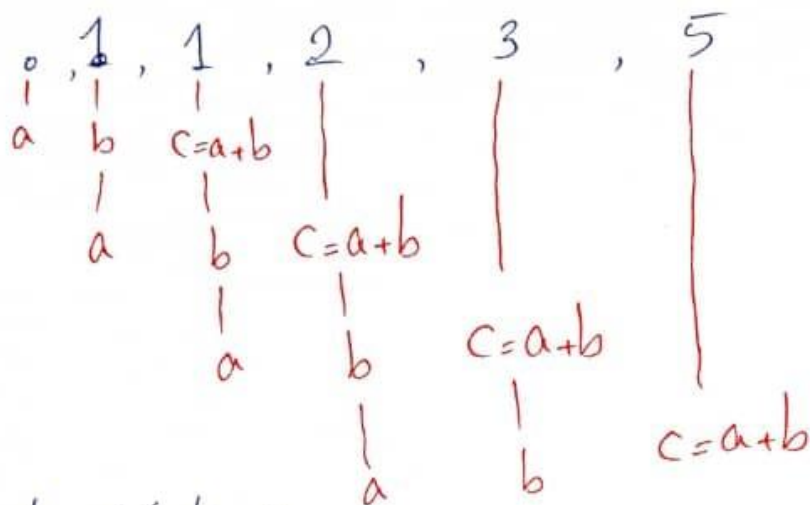
```
    if (n == 1)
        return 1;
```

```
    return F(n-1) + F(n-2);
```

```
}
```



سری فیبوناچی: صورت کلی



```

int F(int n)
{
    int a=0;
    int b=1;
    for(i=2; i<=n; i++)
    {
        c=a+b;
        a=b;
        b=c;
    }
    return c;
}

```

سبب سری فیبوناچی به این

	۰	۱	۲	۳	۴	۵	...
Fib	۰	۱	۱	۲	۳	۵	...

```
int F(int n)
{
```

```
    fib[0] = 0;
```

```
    fib[1] = 1;
```

```
    for (int i = 2; i <= n; i++)
```

```
        fib[i] = fib[i-1] + fib[i-2];
```

```
    return fib[n];
```

```
}
```

؟ روش پویا
Dynamic programming

مسئله ضرب زنجیره‌ای ماتریس‌ها

ضرب ماتریس‌ها

در ضرب ماتریس‌ها قانون کلی به شکل زیر است :

$$A_{m \times n} \times B_{n \times p} = C_{m \times p}$$

به عبارت دیگر باید تعداد ستون‌های ماتریس اول با تعداد سطرهای ماتریس دوم برابر باشد و همچنین تعداد سطرهای ماتریس حاصل برابر با تعداد سطرهای ماتریس اول و تعداد ستون‌های ماتریس حاصل برابر با تعداد ستون‌های ماتریس دوم است.

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

A B C

A, B and C are square matrices of size $N \times N$
 a, b, c and d are submatrices of A, of size $N/2 \times N/2$
 e, f, g and h are submatrices of B, of size $N/2 \times N/2$

Matrix-Matrix Multiplication

– multiplying two $n \times n$ dense, square matrices A and B to yield the product matrix $C = A \times B$.

for simplicity, we use the following serial algorithm:

```

1.  procedure MAT_MULT (A, B, C)
2.  begin
3.      for i := 0 to n - 1 do
4.          for j := 0 to n - 1 do
5.              begin
6.                  C[i, j] := 0;
7.                  for k := 0 to n - 1 do
8.                      C[i, j] := C[i, j] + A[i, k] × B[k, j];
9.                  endfor;
10. end MAT_MULT
  
```

مثال: اگر دو ماتریس A و B به شکل زیر باشند، حاصل ضرب آنها برابر است با:

$$A \begin{bmatrix} 2 & 3 & 5 \\ 1 & 4 & 3 \end{bmatrix}_{2 \times 3} \times B \begin{bmatrix} 6 & 1 \\ 7 & 3 \\ 5 & 2 \end{bmatrix}_{3 \times 2} = \begin{bmatrix} 2 \times 6 + 3 \times 7 + 5 \times 5 & 2 \times 1 + 3 \times 3 + 5 \times 2 \\ 1 \times 6 + 4 \times 7 + 3 \times 5 & 1 \times 1 + 4 \times 3 + 3 \times 2 \end{bmatrix}_{2 \times 2}$$

همانطور که مشاهده می کنید تعداد عمل ضرب در ضرب دو ماتریس $A_{2 \times 3} \times B_{3 \times 2}$ برابر با ۱۲ است.

در ضرب دو ماتریس $A_{m \times n} \times B_{n \times p}$ تعداد عمل ضرب برابر است با: $m \times n \times p$

ضرب زنجیره ای ماتریس ها

ضرب زنجیره ای ماتریس ها دارای خاصیت شرکت پذیری می باشد ولی جابجایی ندارد و علاوه بر این ضرب دو ماتریس A و B در صورتی امکان پذیر است که بعد وسط یکسان باشد یعنی تعداد ستون های ماتریس اول با تعداد سطرهای ماتریس دوم یکسان باشند و همچنین می دانیم که برای ضرب دو ماتریس $A_{m \times n} \times B_{n \times p}$ نیاز به $m \times n \times p$ تعداد ضرب می باشد.

برای ضرب سه ماتریس $A_{m \times n} \times B_{n \times p} \times C_{p \times q}$ دو حالت وجود دارد:

$$(A_{m \times n} \times B_{n \times p}) \times C_{p \times q} \quad m \times n \times p \quad m \times n \times p + m \times p \times q$$

$$10 \times 20 \times 30 \quad 10 \times 20 \times 30 + 10 \times 30 \times 40 = 18000$$

$$A_{m \times n} \times (B_{n \times p} \times C_{p \times q}) \quad m \times n \times q \quad n \times p \times q + m \times n \times q$$

ضرب به ترتیب

۱) $M_1 \times M_2$

دفعه اول ضرب

$$M_1 (3 \times 5) \times M_2 (5 \times 7) = M_1 M_2 (3 \times 7)$$

$$3 \times 5 \times 7 = 105$$

عمل ضرب

۲) $M_1 \times M_2 \times M_3 \rightarrow (M_1 \times M_2) \times M_3$
 $M_1 \times (M_2 \times M_3)$

$$M_1 (3 \times 8) \times M_2 (8 \times 5) \times M_3 (5 \times 2)$$

$$(M_1 (3 \times 8) \times M_2 (8 \times 5)) \times M_3 (5 \times 2)$$

$$3 \times 8 \times 5 = 120$$

عمل ضرب

$$M_1 M_2 (3 \times 5)$$

$$3 \times 5 \times 2 = 30$$

عمل ضرب

$$150 \text{ عمل ضرب}$$

$$M_1_{(3 \times 8)} \times (M_2_{(8 \times 5)} \times M_3_{(5 \times 2)})$$

$$8 \times 5 \times 2 = 80$$

عمل ضرب

$$M_2 M_3_{(8 \times 2)}$$

$$3 \times 8 \times 2 = 48$$

عمل ضرب

$$128$$

عمل ضرب

$$3) M_1 \times M_2 \times M_3 \times M_4$$

$$M_1 \times (M_2 \times M_3 \times M_4) \rightarrow M_1 \times ((M_2 \times M_3) \times M_4)$$

$$\searrow M_1 \times (M_2 \times (M_3 \times M_4))$$

$$(M_1 \times M_2) \times (M_3 \times M_4)$$

$$(M_1 \times M_2 \times M_3) \times M_4 \rightarrow ((M_1 \times M_2) \times M_3) \times M_4$$

$$\searrow (M_1 \times (M_2 \times M_3)) \times M_4$$

بررسی و تأیید

فبرل کاتالان

$$\frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{(n+1)! n!}$$

$$n=3 \quad \frac{1}{3+1} \binom{2 \times 3}{3} = \frac{1}{4} \binom{6}{3} = \frac{1}{4} \frac{6!}{3! (6-3)!} = \frac{1}{4} \frac{6!}{3! 3!}$$

$$= \frac{6 \times 5 \times 4 \times 3!}{4 \times 3! \times 3!} = 5$$

$$n=4 \quad \frac{(2 \times 4)!}{(4+1)! 4!} = \frac{8!}{5! 4!} = \frac{8 \times 7 \times 6 \times 5!}{5! \times 4!} = 14$$

اعداد کاتالان: 1, 2, 5, 14, 42, 132, 429, 1430, 4862, ...

نکته: برای محاسبه تعداد حالت‌ها ضرب n تریس دریم؛ یعنی یک واحد کمتر

از فبرل کاتالان قرار دیم

$$n=2 \quad \text{ضرب 3 تریس} \quad \text{تعداد} = 2$$

$$n=3 \quad \text{ضرب 4 تریس} \quad \text{تعداد} = 5$$

$$n=4 \quad \text{ضرب 5 تریس} \quad \text{تعداد} = 14$$

هدف ما این است که در بین تمامی این حالت‌ها، حالتی را بیابیم که حاصلضرب ماتریس‌ها با حداقل ضرب عددی محاسبه شود.

استفاده از روش تقسیم و حل

ابتدا سعی می‌کنیم با استفاده از روش تقسیم و حل الگوریتمی برای پاسخ مسئله پیدا کنیم. برای این کار عبارت مورد نظر را به دو قسمت تقسیم می‌کنیم. به عنوان مثال فرض کنید $n = 7$ باشد. در این صورت به شش طریق می‌توان عبارت را به دو دسته تقسیم کرد:

- 1: $(M1) \times (M2 \times M3 \times M4 \times M5 \times M6 \times M7)$
- 2: $(M1 \times M2) \times (M3 \times M4 \times M5 \times M6 \times M7)$
- 3: $(M1 \times M2 \times M3) \times (M4 \times M5 \times M6 \times M7)$
- 4: $(M1 \times M2 \times M3 \times M4) \times (M5 \times M6 \times M7)$
- 5: $(M1 \times M2 \times M3 \times M4 \times M5) \times (M6 \times M7)$
- 6: $(M1 \times M2 \times M3 \times M4 \times M5 \times M6) \times (M7)$

اگر در فرمول کاتالان به جای $n=6$ قرار دهیم، تعداد حالت پُرانتز گذاری برای ضرب ۷ ماتریس ۱۳۲ حالت خواهد شد.

هر کدام از حاصلضرب‌های داخل پُرانتزها، زیرمسئله‌ای با $n > 7$ هستند. همانطور که می‌دانید در ضرب دو ماتریس، تعداد ستون‌های ماتریس اول با تعداد سطرهای ماتریس دوم برابر است. در نتیجه ابعاد هفت ماتریس فوق را به صورت زیر می‌توان خلاصه کرد:

$d_0, d_1, d_2, d_3, d_4, d_5, d_6, d_7$

که ماتریس اول d_0 سطر و d_1 ستون، ماتریس دوم d_1 سطر و d_2 ستون ... و ماتریس هفتم d_6 سطر و d_7 ستون دارد. حال تابع Mult را به گونه‌ای تعریف می‌کنیم که حداقل ضرب‌های مورد نیاز برای ضرب ماتریس‌های d_i تا d_j را محاسبه کند. در این صورت برای شش حالت فوق داریم:

- 1: $(M1) \times (M2 \times M3 \times M4 \times M5 \times M6 \times M7): \min 1 = \text{Mult}(1, 1) + \text{Mult}(2, 7) + d_0 d_1 d_7$
- 2: $(M1 \times M2) \times (M3 \times M4 \times M5 \times M6 \times M7): \min 2 = \text{Mult}(1, 2) + \text{Mult}(3, 7) + d_0 d_2 d_7$
- 3: $(M1 \times M2 \times M3) \times (M4 \times M5 \times M6 \times M7): \min 3 = \text{Mult}(1, 3) + \text{Mult}(4, 7) + d_0 d_3 d_7$
- 4: $(M1 \times M2 \times M3 \times M4) \times (M5 \times M6 \times M7): \min 4 = \text{Mult}(1, 4) + \text{Mult}(5, 7) + d_0 d_4 d_7$
- 5: $(M1 \times M2 \times M3 \times M4 \times M5) \times (M6 \times M7): \min 5 = \text{Mult}(1, 5) + \text{Mult}(6, 7) + d_0 d_5 d_7$
- 6: $(M1 \times M2 \times M3 \times M4 \times M5 \times M6) \times (M7): \min 6 = \text{Mult}(1, 6) + \text{Mult}(7, 7) + d_0 d_6 d_7$

قسمت اول عبارت‌های فوق مربوط به محاسبه‌ی زیرمسئله‌ی سمت چپ، قسمت دوم برای محاسبه‌ی زیرمسئله‌ی سمت راست و قسمت سوم ضرب دو ماتریس حاصل از زیرمسئله‌ها در یکدیگر است. به عنوان نمونه، در سطر سوم پس از تفکیک مسئله به دو زیرمسئله، دو ماتریس با ابعاد (d^4, d^3) و (d^3, d^7) به دست می‌آیند که باید در پایان کار در هم ضرب شوند. حال با مینیمم گرفتن از $\min_k$ ها مسئله‌ی اصلی حل می‌شود:

$$\text{Mult}(1,7)=\min\{mink\}=\min\{\text{Mult}(1,k)+\text{Mult}(k+1,7)+d_0d_kd_7\},k=1,\dots,6$$

این رابطه را می‌توان به صورت کلی برای هر شروع و انتهایی نوشت:

$$\text{Mult}(i,j)=\min\{mink\}=\min\{\text{Mult}(i,k)+\text{Mult}(k+1,j)+d_{i-1}d_kd_j\},k=i,\dots,j-1$$

توجه داشته باشید که اگر $i > j$ نباشد منطق حکم می‌کند که $\text{Mult}(i,j)$ صفر شود. با توجه به توضیحات فوق، بدنه‌ی تابع Mult در حالت کلی به این صورت خواهد بود:

```
int Mult(int i, int j){
    int min;
    min = minimum of { Mult(i, k) + Mult(k + 1, j) + d[i - 1] * d[k] * d[j] } for k = i, i + 1, i + 2, . . . , j - 1;
    return min;
}
```

در این تابع ابعاد ماتریس‌ها در فرم آرایه‌ی d به صورت عمومی تعریف شده‌اند. ثابت شده است مرتبه‌ی اجرای چنین تابعی نمایی است. به نظر می‌رسد که این روش کارا نیست. علت آن هم وجود همپوشانی است که در ادامه به آن می‌پردازیم.

استفاده از روش برنامه‌نویسی پویا

۲- اصل بهینگی: اگر بنا باشد پراتزبندی کل عبارت بهینه شود، پراتزبندی زیرمسئله‌ها هم باید بهینه باشند. یعنی بهینه بودن مسئله، بهینه بودن زیرمسئله‌ها را ایجاب می‌کند.

پس می‌توان از روش برنامه‌نویسی پویا استفاده کرد.

در روش برنامه‌نویسی پویا اغلب از یک آرایه برای ذخیره نتایج جهت استفاده‌ی مجدد استفاده شده و زیرمسائل به صورت جزء به کل حل می‌شوند. در مورد این مسئله، ابتدا زیرمسائلی که تنها از دو ماتریس تشکیل شده‌اند محاسبه

می شوند. سپس زیرمسائلی که از سه ماتریس تشکیل شده اند محاسبه می شوند. این دسته از زیرمسائل به زیرمسائلی متشکل از دو ماتریس تجزیه می شوند که قبلاً محاسبات آنها صورت گرفته و نتایج آنها در آرایه ذخیره شده اند. در نتیجه نیاز به محاسبه ی مجدد آنها نیست. به همین ترتیب در مرحله ی بعد زیرمسائل با چهار ماتریس محاسبه می شوند و الی آخر.

برای پیاده سازی این روش، آرایه ی دو بعدی M را منطبق با عملکرد تابع Mult تعریف می کنیم:

$$1 \quad M[i][j] = \text{Min} \{ M[i][k] + M[k+1][j] + d[i] * d[k+1] * d[j] \}$$

$$2 \quad i \leq k \leq j-1, 0 < i < j < n+1, M[i][i] = 0$$

توجه داشته باشید که اندیس شروع آرایه ها را یک در نظر گرفته ایم و تنها از قسمت بالای قطر اصلی ماتریس استفاده می کنیم. تابع زیر محاسبات لازم را جهت ساخت این ماتریس انجام می دهد:

```

1 int Mult(int n){
2   int i, j;
3   for(i = 1 ; i <= n ; i++)
4     M[i][i] = 0;
5   for(i = 1 ; i < n ; i++)
6     for(j = 1 ; j <= n - i ; j++)
7       M[j][i+j] = Minimum of { M[i][k] + M[k+1][j] + d[i-1] * d[k] * d[j] } for k =
        i, i+1, i+2, ..., j-1
8   return M[1][n];
9 }
```

تابع مقدار $M[1][n]$ را باز می گرداند که همان تعداد ضربهای لازم جهت محاسبه ی ضرب زنجیره ای n ماتریس در حالت پرانتز بندی بهینه است. این تابع دو حلقه ی تکرار دارد که مرتبه ی $O(n^2)$ را نشان می دهند. اما یافتن مینیمم داده ها درون حلقه ی داخلی نیز وجود دارد که از مرتبه ی $O(n)$ است. در نتیجه مرتبه ی کل تابع فوق $O(n^3)$ خواهد بود که در مقایسه با مرتبه نمایی بهبود چشم گیری را نشان می دهد.

مزیت دیگر این روش - که از اصل بهینگی ناشی می شود - این است که با انجام این محاسبات، نه تنها مسئله اصلی که حالت بهینه‌ی همه‌ی زیرمسائل حل شده است. به عنوان نمونه، $M[۲][۴]$ تعداد ضرب‌های لازم در پرانتزبندی بهینه‌ی ماتریس‌های دوم تا چهارم را نشان می‌دهد.

در پایان باید توجه داشته باشید که:

۱- در دو تابعی که اینجا آورده شده است، تنها تعداد ضرب‌های لازم در پرانتزبندی بهینه محاسبه گشته و شیوه‌ی پرانتزبندی مد نظر نیست.

۲- به نظر می‌رسد روش برنامه‌نویسی پویا از روش تقسیم و حل قدرتمندتر است! اما همیشه اینگونه نیست. ما در همه حال باید به دو شرط اساسی امکان استفاده از برنامه‌نویسی پویا جهت طراحی الگوریتم توجه داشته باشیم. در بسیاری از مسائل این دو شرط برقرار نیستند و ممکن است طراحی الگوریتم حل آنها با استفاده از روش تقسیم و حل کاراتر باشد.

ضرب زنجیره ای ماتریس ها

در این مسئله فرض می کنیم ماتریس M_i دارای ابعاد $r_{i-1} \times r_i$ می باشد و

m_{ij} حداقل تعداد عملیات ضرب عددی برای محاسبه $M_i \times M_{i+1} \times \dots \times M_j$ می باشد و $m_{ii} = 0$.

برای محاسبه m_{ij} خواهیم داشت: $(M_i \times \dots \times M_k) \times (M_{k+1} \times \dots \times M_j)$

بنابراین m_{ij} از رابطه زیر بدست می آید:

$$m_{ij} = \min_{i \leq k \leq j} (m_{ik} + m_{k+1,j} + r_{i-1} \times r_k \times r_j)$$

الگوریتم ضرب زنجیره ای ماتریس ها

مرحله ۰: هیچ عمل ضرب ماتریسی. این مرحله معادل یادداشت $m_{ii} = 0$ برای $i = 1, 2, \dots, n$ می باشد.

مرحله ۱: یک عمل ضرب ماتریسی. در این مرحله کلیه $m_{i,i+1}$ برای $i = 1, 2, \dots, n$ را محاسبه و در

جدول قرار می دهیم.

مرحله ۲: دو عمل ضرب ماتریسی. در این مرحله کلیه $m_{i,i+2}$ برای $i = 1, 2, \dots, n$ را محاسبه و در جدول

قرار می دهیم.

مرحله $n-1 : n-1$ عمل ضرب ماتریسی. در این مرحله m_{1n} که همان جواب است را محاسبه می کنیم.

مثال ضرب زنجیره ای ماتریس ها

$$A_{1 \times 2} \times B_{2 \times 5} \times C_{5 \times 1} \times D_{1 \times 10}$$

$$m_{1,1} = m_{2,2} = m_{3,3} = m_{4,4} = 0$$

$$m_{1,2} = \min(m_{1,1} + m_{2,2} + 1 \times 2 \times 5) = 10$$

$$m_{2,3} = \min(m_{2,2} + m_{3,3} + 2 \times 5 \times 1) = 10$$

$$m_{3,4} = \min(m_{3,3} + m_{4,4} + 5 \times 1 \times 10) = 50$$

$$m_{13} = \min(m_{11} + m_{23} + 10 \times 20 \times 1, m_{12} + m_{33} + 10 \times 50 \times 1) = 1200$$

$$m_{24} = \min(m_{22} + m_{34} + 20 \times 50 \times 100, m_{23} + m_{44} + 20 \times 1 \times 100) = 3000$$

$$m_{14} = \min(m_{11} + m_{24} + 10 \times 20 \times 100, m_{12} + m_{34} + 10 \times 50 \times 100, m_{13} + m_{44} + 10 \times 1 \times 100) = 2200$$

مثال ضرب زنجیره ای ماتریس ها

$$\begin{aligned}
 m_{11} &= 0 & m_{22} &= 0 & m_{33} &= 0 & m_{44} &= 0 \\
 m_{12} &= 10000 & m_{23} &= 1000 & m_{34} &= 5000 \\
 m_{13} &= 1200 & m_{24} &= 3000 \\
 m_{14} &= 2200
 \end{aligned}$$

$$\begin{bmatrix}
 0 & 10000 & 1200 & 2200 \\
 & 0 & 1000 & 3000 \\
 & & 0 & 5000 \\
 & & & 0
 \end{bmatrix}$$

دقت شود که در ضرب چهار ماتریس در هم ۵ حالت بوجود آمد که برای انجام هر کدام نیاز به ۳ واحد ضربی است که در مجموع ۱۵ عمل ضرب نیاز است در حالیکه در روش بهینه فقط نیاز به ۱۰ واحد ضربی است. (واحد ضربی همان ضرب دو ماتریس در هم در نظر گرفته شود)

$$m_{1,r} = \min(m_{1,1} + m_{r,r} + 1 \times 20 \times 50) = 10000$$

$$m_{r,r} = \min(m_{r,r} + m_{r,r} + 1 \times 50 \times 1) = 1000$$

$$m_{r,r} = \min(m_{r,r} + m_{r,r} + 50 \times 1 \times 100) = 5000$$

$$m_{13} = \min(m_{11} + m_{23} + 10 \times 20 \times 1, m_{12} + m_{33} + 10 \times 50 \times 1) = 1200$$

$$m_{24} = \min(m_{22} + m_{34} + 20 \times 50 \times 100, m_{23} + m_{44} + 20 \times 1 \times 100) = 3000$$

$$m_{14} = \min(m_{11} + m_{24} + 10 \times 20 \times 100, m_{12} + m_{34} + 10 \times 50 \times 100, m_{13} + m_{44} + 10 \times 1 \times 100) = 2200$$

اگر به ضرب ۷ ماتریس در هم رجوع کنیم ۱۳۲ حالت ایجاد شد که برای محاسبه هر کدام از حالت ها نیاز به ۶ واحد ضربی است که در مجموع باید $792 = 132 \times 6$ واحد ضربی نیاز است. در حالیکه در روش بهینه فقط به ۵۶ واحد ضربی نیاز است

$$56 = 6 + 5 \times 2 + 4 \times 3 + 3 \times 4 + 2 \times 5 + 6$$

واحد ضربی نیاز است.

ضرب ماتریس های بزرگ با استفاده از الگوریتم استراسن

دو ماتریس مربعی A و B با اندازه های $n \times n$ داده شده است. هدف، پیدا کردن ماتریس حاصل ضرب این دو ماتریس است.

روش ساده ضرب ماتریس ها

در ادامه، یک راهکار ساده برای ضرب دو ماتریس داده شده است.

```
void multiply(int A[][N], int B[][N], int C[][N])
{
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            C[i][j] = 0;
            for (int k = 0; k < N; k++)
            {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}
```

پیچیدگی زمانی روش بالا از درجه $O(n^3)$ است.

روش تقسیم و حل ضرب ماتریس ها

در ادامه، مراحل یک روش تقسیم و حل ساده برای ضرب کردن دو ماتریس مربعی بیان شده است.

تقسیم ماتریس های A و B در ۴ زیر ماتریس با اندازه $N/2 * N/2$ به صورتی که در تصویر زیر نمایش داده شده است.

محاسبه مقادیری که در ادامه آمده است، به صورت بازگشتی .

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

A B C

A، B و C ماتریس‌های مربعی در اندازه $N \times N$ هستند.
 a، b و c زیرماتریس‌های A با اندازه $N/2 \times N/2$ هستند.
 e، f و g زیرماتریس‌های B با اندازه $N/2 \times N/2$ هستند.

در روش بالا، هشت ضرب برای ماتریس‌هایی از اندازه $N/2 * N/2$ و ۴ جمع انجام شده است. جمع کردن دو ماتریس، زمانی از درجه $O(n^2)$ دارد. بنابراین، پیچیدگی زمانی به صورت زیر نوشته می‌شود.

$$T(n) = 8T(n/2) + O(n^2)$$

با توجه به «قضیه اصلی واکاوی الگوریتم‌ها» (Master's Theorem) پیچیدگی زمانی الگوریتم بالا از درجه $O(N^3)$ و متأسفانه، برابر با روش ساده‌ای است که در ابتدای این مطلب معرفی شد.

الگوریتم استراسن برای ضرب ماتریس‌ها

در روش تقسیم و حل ارائه شده در بالا، عامل اصلی که منجر به پیچیدگی زمانی بالایی می‌شود، ۸ فراخوانی بازگشتی است. ایده الگوریتم استراسن، کاهش تعداد فراخوانی‌های بازگشتی به ۷ تا است. روش استراسن از این جهت مشابه با روش ساده تقسیم و حل ارائه شده در بالا است که ماتریس‌ها را به زیرماتریس‌هایی با ابعاد $N/2 \times N/2$ تقسیم می‌کند. این مورد، در تصویر زیر به خوبی به نمایش داده شده است. اما در الگوریتم استراسن، چهار زیرماتریس از نتیجه با استفاده از فرمولی که در ادامه آمده است محاسبه می‌شوند.

ضرب ماتریس‌های استراسن (Strassen's Matrix Multiplication)

فرض کنید که می‌خواهیم ماتریس C که حاصل ضرب دو ماتریس ۲×۲ A و B است را بدست آوریم.

$$\begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

استراسن فرمول‌هایی به صورت زیر ارائه کرده‌است که با بهره‌گیری از آنها می‌توان ماتریس C را محاسبه نمود.

$$m_1 = (a_{11} + a_{22})(b_{11} + b_{22}) \text{ if we let}$$

$$m_2 = (a_{21} + a_{22})b_{11}$$

$$m_3 = a_{11}(b_{12} - b_{22})$$

$$m_4 = a_{22}(b_{21} - b_{11})$$

$$m_5 = (a_{11} + a_{12})b_{22}$$

$$m_6 = (a_{21} - a_{11})(b_{11} + b_{12})$$

$$m_7 = (a_{12} - a_{22})(b_{21} + b_{22}),$$

$$C = \begin{bmatrix} m_1 + m_4 - m_5 + m_7 & m_3 + m_5 \\ m_2 + m_4 & m_1 + m_3 - m_2 + m_6 \end{bmatrix}$$

ضرب ماتریس های استراسن

- این روش برای ماتریس های 2×2 چندان جالب نیست
- فرمول های استراسن به گونه ای هستند که می توانیم ماتریس های بزرگتر را به صورت افراز ۴ ماتریس کوچکتر در نظر گرفت

$$\begin{matrix} \leftarrow n/2 \rightarrow \\ \uparrow n/2 \downarrow \end{matrix} \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$A_{11} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1,n/2} \\ a_{21} & a_{22} & \cdots & a_{2,n/2} \\ \vdots & & & \\ a_{n/2,1} & \cdots & a_{n/2,n/2} \end{bmatrix} \quad M_1 = (A_{11} + A_{22})(B_{11} + B_{22}),$$

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

ضرب ماتریس‌های استراسن

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \end{bmatrix} \quad B = \begin{bmatrix} 8 & 9 & 1 & 2 \\ 3 & 4 & 5 & 6 \\ 7 & 8 & 9 & 1 \\ 2 & 3 & 4 & 5 \end{bmatrix}$$

$$\begin{array}{c} \leftarrow 2 \rightarrow \\ \uparrow 2 \downarrow \end{array} \begin{bmatrix} & & & \\ C_{11} & & C_{12} & \\ & & & \\ C_{21} & & C_{22} & \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \end{bmatrix} \times \begin{bmatrix} 8 & 9 & 1 & 2 \\ 3 & 4 & 5 & 6 \\ 7 & 8 & 9 & 1 \\ 2 & 3 & 4 & 5 \end{bmatrix}$$

ضرب ماتریس های استراسن

$$m_1 = (a_{11} + a_{22})(b_{11} + b_{22})$$

$$m_2 = (a_{21} + a_{22})b_{11}$$

$$m_3 = a_{11}(b_{12} - b_{22})$$

$$m_4 = a_{22}(b_{21} - b_{11})$$

$$m_5 = (a_{11} + a_{12})b_{22}$$

$$m_6 = (a_{21} - a_{11})(b_{11} + b_{12})$$

$$m_7 = (a_{12} - a_{22})(b_{21} + b_{22})$$

$$C = \begin{bmatrix} m_1 + m_4 - m_5 + m_7 & m_3 + m_5 \\ m_2 + m_4 & m_1 + m_3 - m_2 + m_6 \end{bmatrix}$$

$$M_1 = (A_{11} + A_{22}) \times (B_{11} + B_{22})$$

$$= \left(\begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} + \begin{bmatrix} 2 & 3 \\ 6 & 7 \end{bmatrix} \right) \times \left(\begin{bmatrix} 8 & 9 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 9 & 1 \\ 4 & 5 \end{bmatrix} \right)$$

$$= \begin{bmatrix} 3 & 5 \\ 11 & 13 \end{bmatrix} \times \begin{bmatrix} 17 & 10 \\ 7 & 9 \end{bmatrix}$$

ضرب ماتریس های استراسن

```

function C=sterasen(A,B)
    n=max(size(A));
    if (n==2)
        C=A*B;
    else
        A11=A(1:n/2,1:n/2);
        A12=A(1:n/2,n/2+1:n);
        A21=A(n/2+1:n,1:n/2);
        A22=A(n/2+1:n,n/2+1:n);
        B11=B(1:n/2,1:n/2);
        B12=B(1:n/2,n/2+1:n);
        B21=B(n/2+1:n,1:n/2);
        B22=B(n/2+1:n,n/2+1:n);

        M1=sterasen(A11+A22,B11+B22);
        M2=sterasen(A21+A22,B11);
        M3=sterasen(A11,B12-B22);
        M4=sterasen(A22,B21-B11);
        M5=
        M6
        M7
        C11=M1+M4-M5+M7;
        C12
        C21
        C22
        C(1:n/2,1:n/2)=C11;
        C(1:n/2,n/2+1:n)=C12;
        C(n/2+1:n,1:n/2)=C21;
        C(n/2+1:n,n/2+1:n)=C22;

    end
end

```

پیچیدگی زمانی جمع و تفریق دو ماتریس به اندازه $O(n^2)$ زمان می برد. بنابراین، پیچیدگی زمانی را می توان به صورت زیر نوشت.

$$T(n) = 7T(n/2) + O(n^2)$$

با توجه به قضیه اساسی، پیچیدگی زمانی الگوریتم بالا از درجه $O(n^{\log^v})$ و تقریباً برابر با $O(n^{2.8})$ است. عموماً، از الگوریتم استراسن به دلایلی که در ادامه بیان شده است، در کاربردهای جهان واقعی استفاده نمی شود

در نتیجه مرتبه ای اجرای الگوریتم به $O(n^{2.8})$ تبدیل می شود.

به عنوان مثال اگر n برابر ۱۰۲۴ باشد در واقع بخواهیم دو ماتریس 1024×1024 را در هم ضرب کنیم :

$$A_{1024 \times 1024} * B_{1024 \times 1024} = 1024 * 1024 * 1024 = 2^{30} = 1/073/741/824$$

یا

$$n = 1024 = 2^{10} \Rightarrow k = 10 \quad n = 1024 = 2^{10} \Rightarrow k = 10$$

$$T(1024) = 8^{10} = 1/073/741/824$$

به روش استراسن

$$T'(1024) = 7^{10} = 282/475/249$$

یعنی در این حالت زمان محاسبه ای حاصل ضرب به روش استراسن نسبت به حالت عادی نزدیک به چهار برابر کمتر می شود.

برای یافتن کوتاه ترین مسیر (Dijkstra) الگوریتم دایجسترا

فرض می شود که یک گراف به همراه یک راس مبدا داده شده و هدف پیدا کردن کوتاه ترین مسیر به همه

راس های موجود در گراف مذکور است. الگوریتم دایجسترا برای «درخت پوشای کمین (Minimum

Spanning Tree) دارد. در الگوریتم دایجسترا نیز درخت کوتاه ترین مسیر با استفاده از مبدا داده شده به

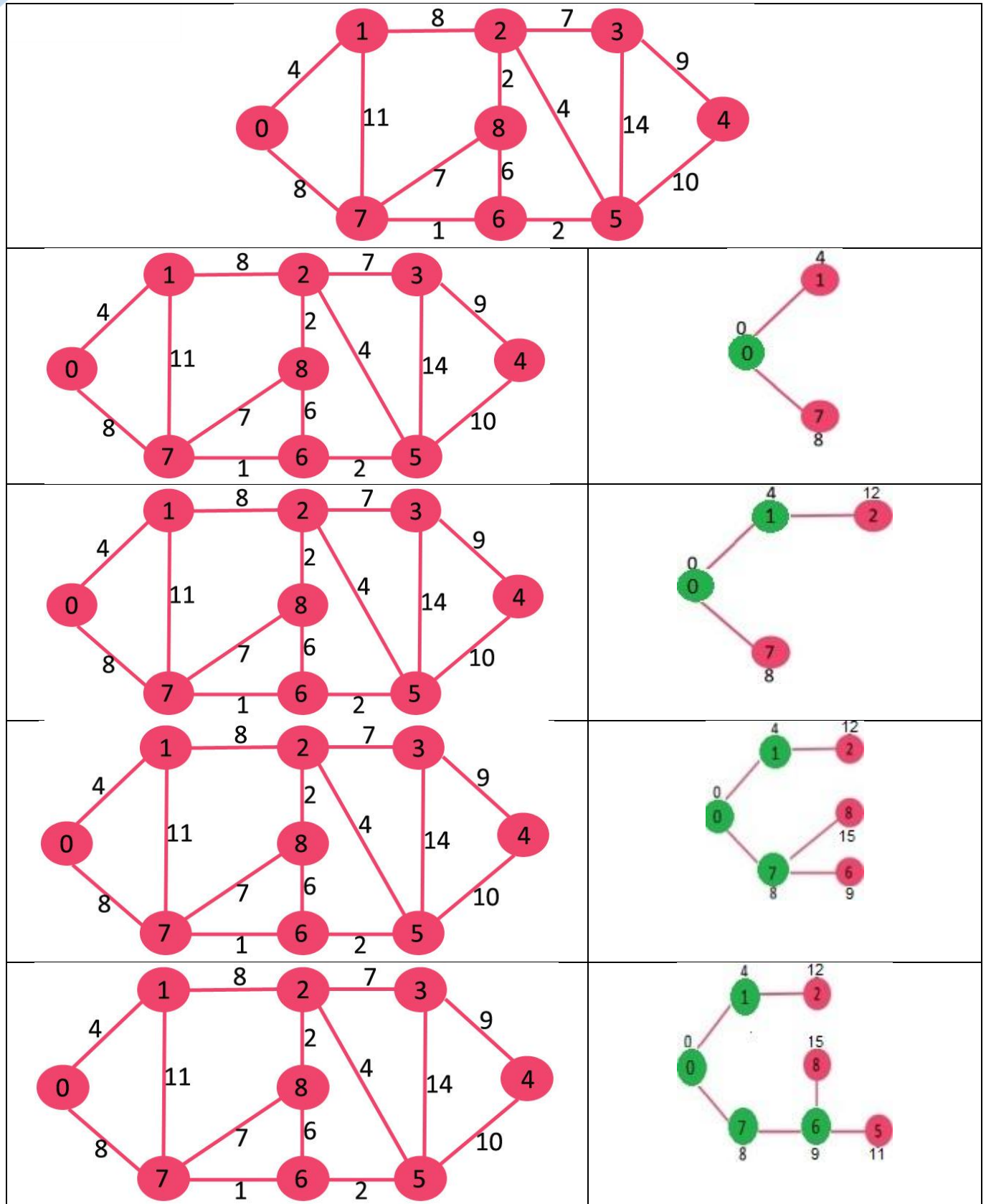
عنوان ریشه، ساخته می شود. در هر مرحله از الگوریتم، راسی پیدا می شود که در مجموعه دیگر (مجموعه

راس های در نظر گرفته نشده) قرار دارد و دارای کمترین فاصله از ریشه است. در ادامه، گام های مورد استفاده

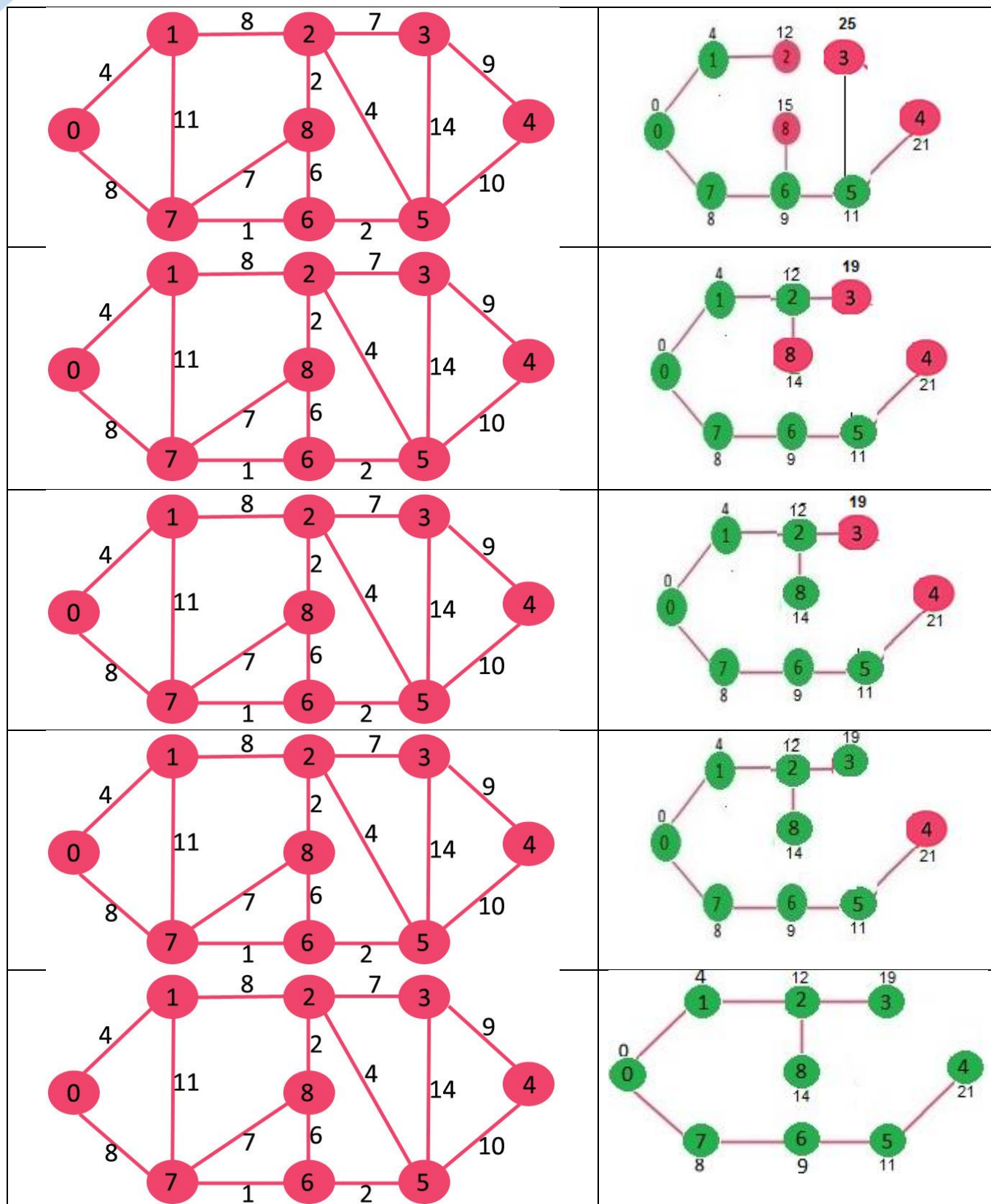
در الگوریتم دایجسترا به منظور یافتن کوتاه ترین مسیر از یک راس مبدا مجرد به دیگر راس ها در گراف داده

شده به صورت مشروح بیان شده اند.

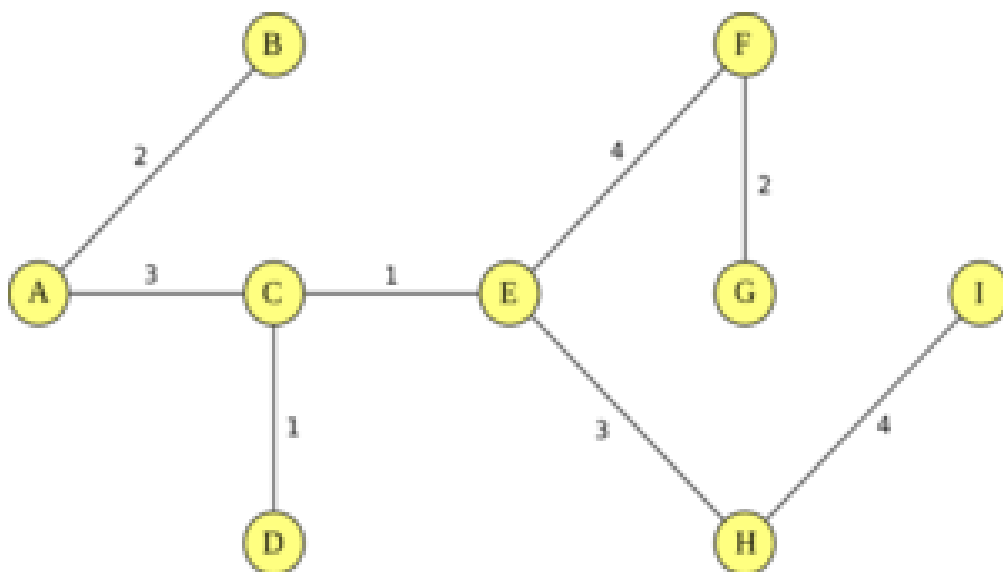
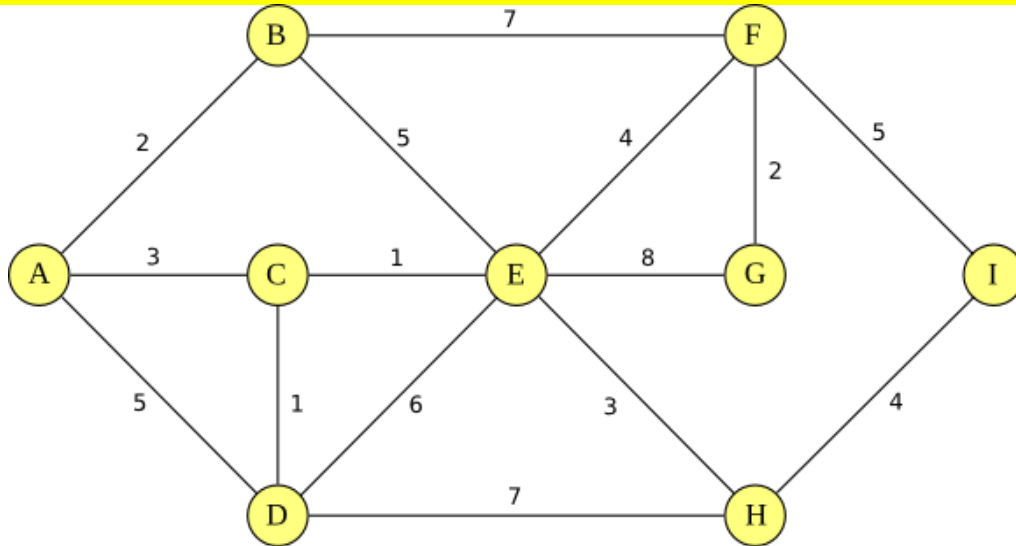
برای درک بهتر موضوع، مثال زیر مورد بررسی قرار خواهد گرفت



طراحی الگوریتم تدوین: دکتر عادل نیا



مثال



الگوریتم کروسکال

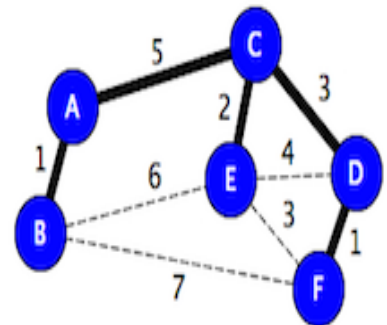
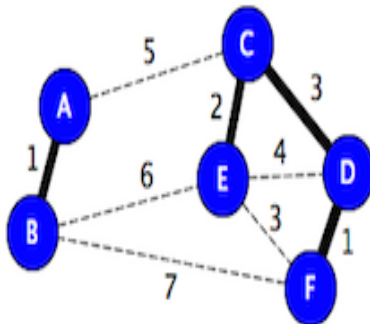
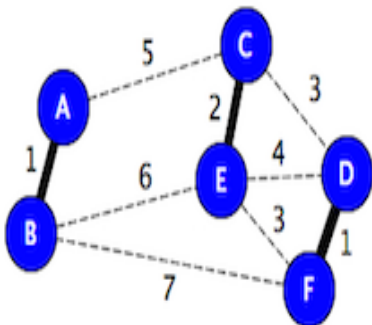
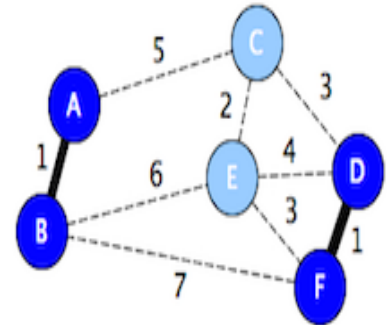
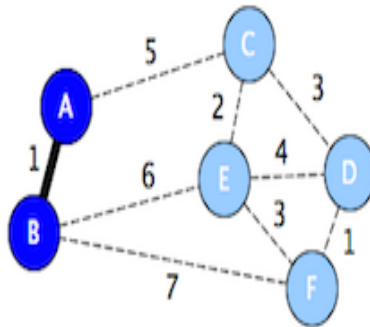
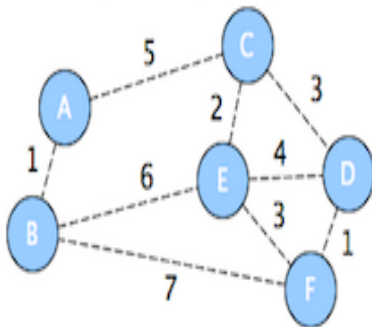
تعریف

الگوریتم کروسکال، الگوریتم برای پیدا کردن درخت پوشای کمینه یک گراف وزن دار است. در واقع در نهایت یک درخت با کمترین هزینه را تشکیل می دهد.

الگوریتم

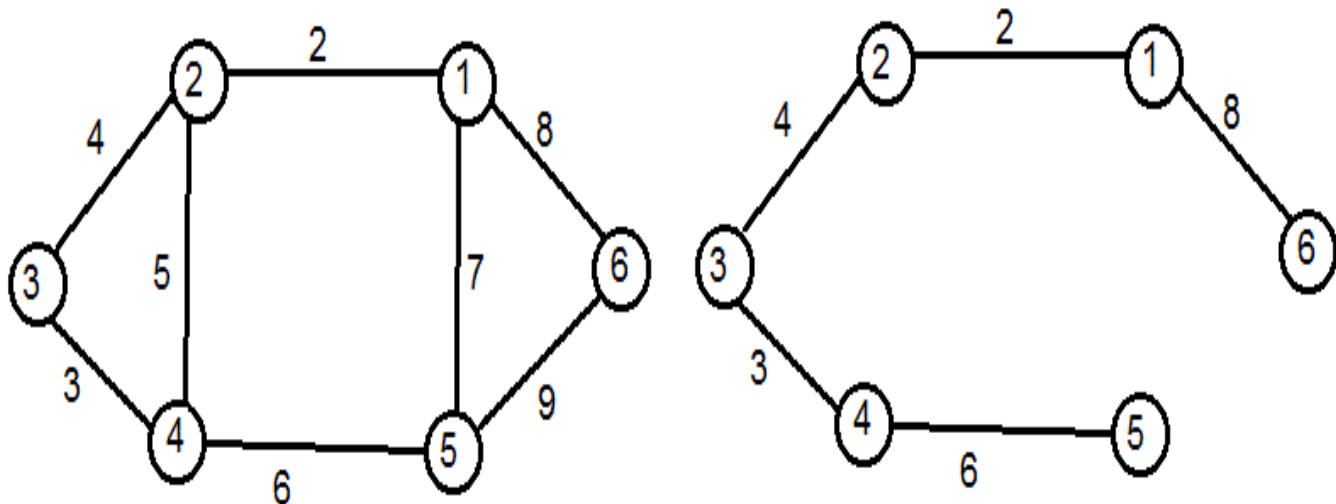
در این الگوریتم، یال ها را بر اساس وزن به صورت صعودی مرتب می کنیم. سپس از اولین یال شروع می کنیم (دارای کمترین هزینه) و هر یالی که با یال هایی که قبلا انتخاب کردیم دور نمی سازد را انتخاب می کنیم. تا جایی که درخت تشکیل شود. در واقع برای n گره، $n-1$ یال را انتخاب می کنیم.

مثال ۱

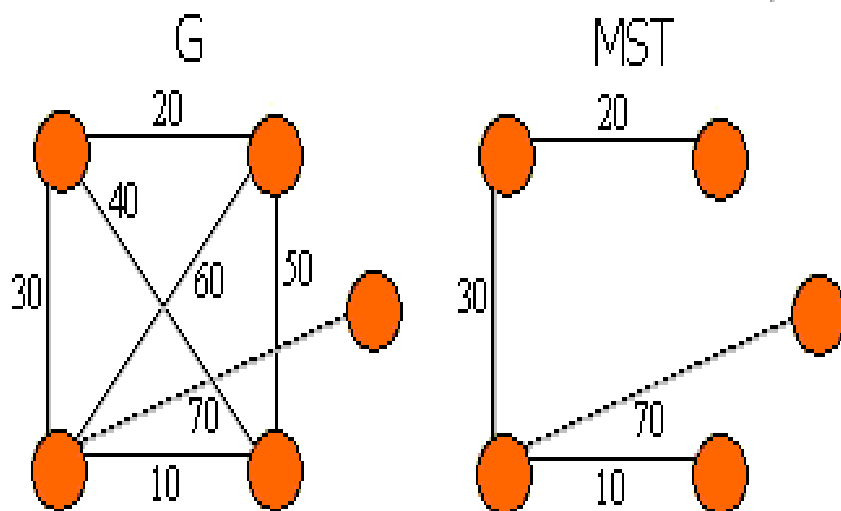


مثال ۲

درخت پوشای کمینه



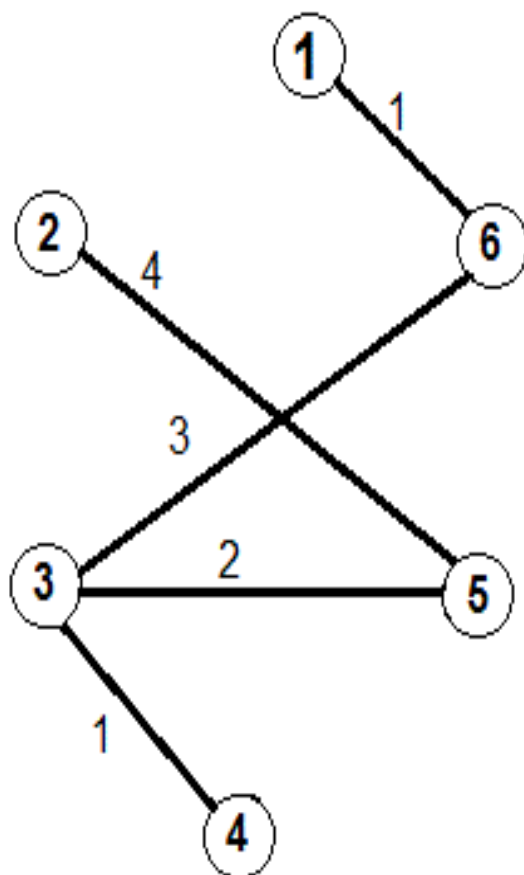
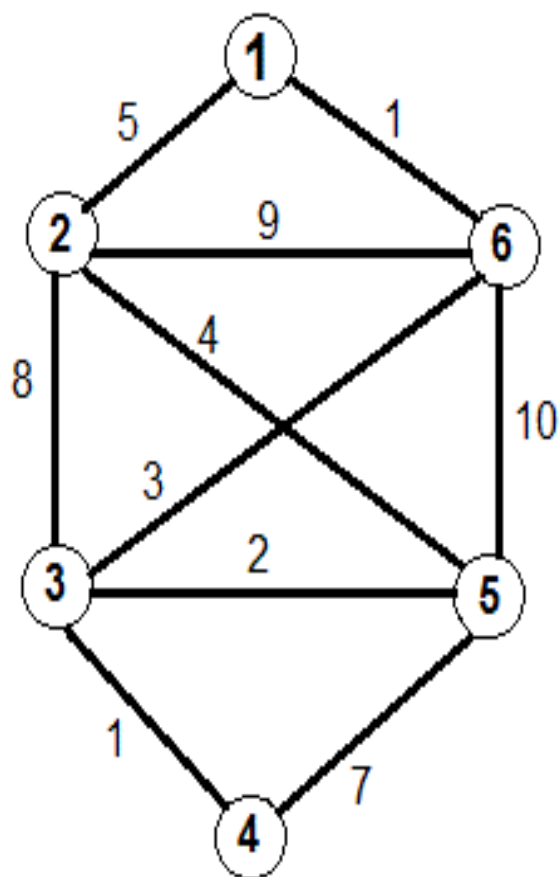
مثال ۳



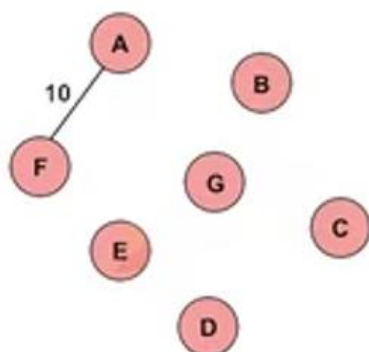
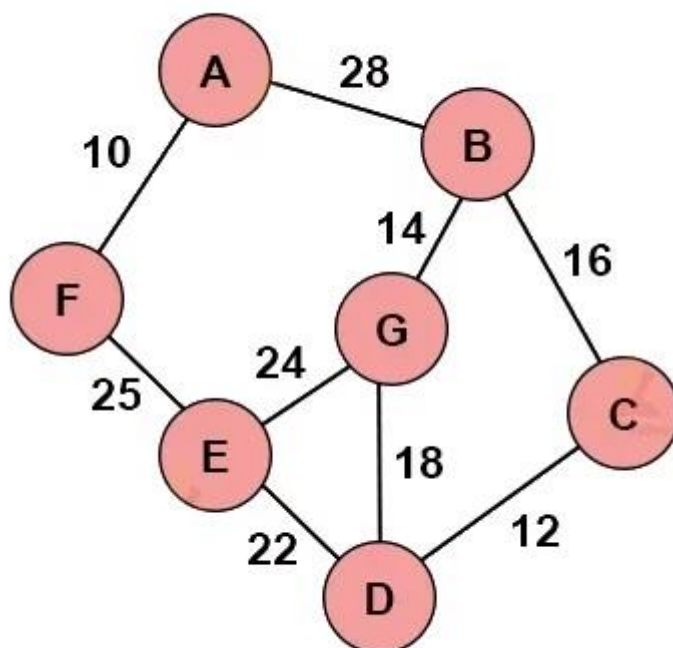
جمع هزینه = 130

مثال ۴

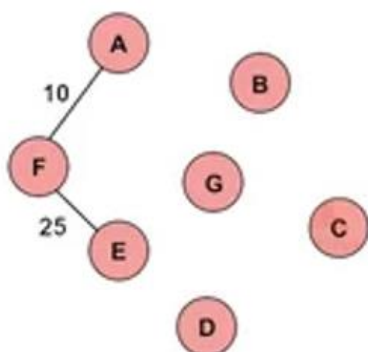
درخت پوشای کمینه



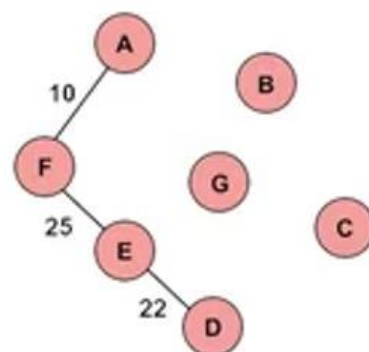
پیدا کردن درخت پوشا با الگوریتم Prim



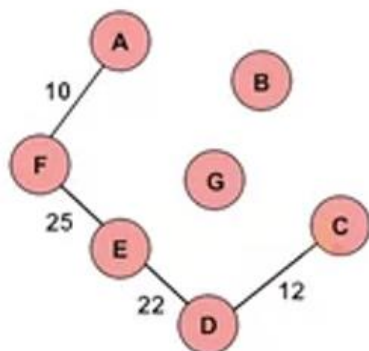
مرحله اول



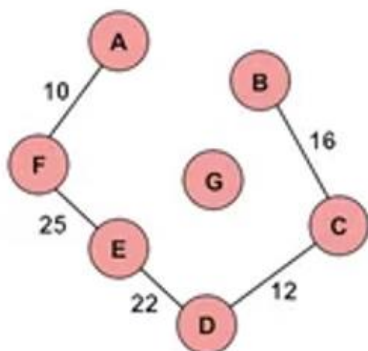
مرحله دوم



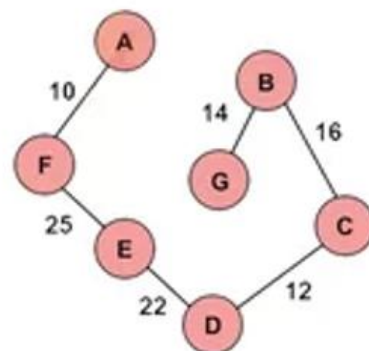
مرحله سوم



مرحله چهارم

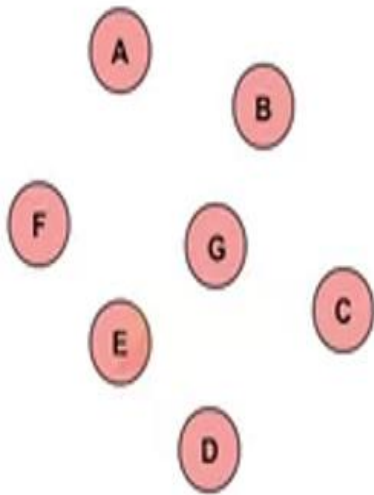


مرحله پنجم

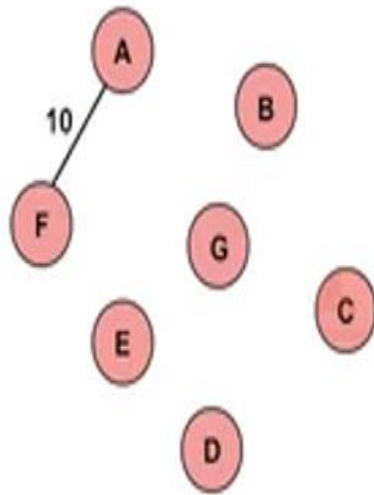


مرحله ششم

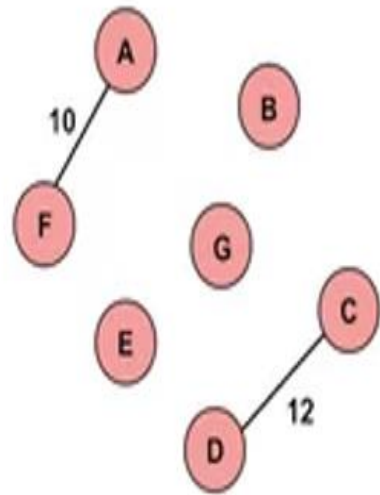
به روش کروسکال



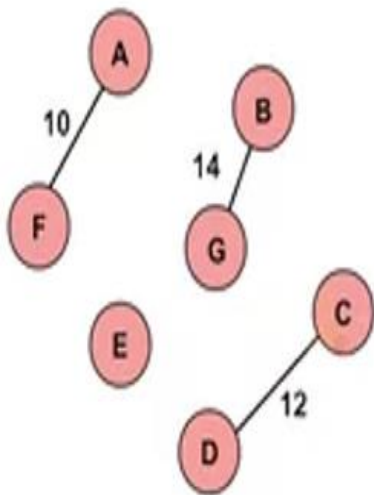
مرحله اول



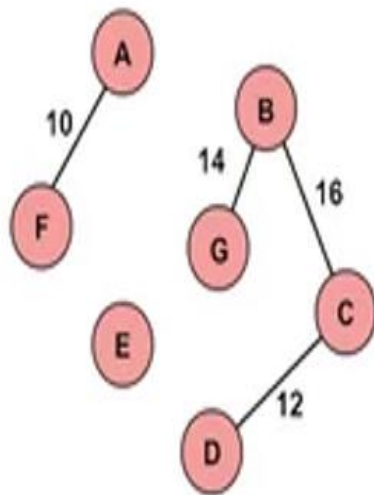
مرحله دوم



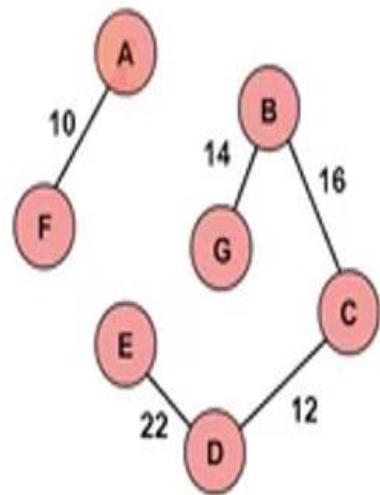
مرحله سوم



مرحله چهارم



مرحله پنجم



مرحله ششم

ضرب اعداد بزرگ

در ضرب دو عدد m, n رقمی نیز به mn عمل ضرب و تعداد سِفْت $O(n^2)$.

$$\begin{array}{r}
 1111 \times \\
 \underline{2222} \\
 2222 \\
 2222 \\
 2222 \\
 2222 \\
 \hline
 2468642
 \end{array}$$

۱ ضرب

سِفْت ۱ و ۱ ضرب

سِفْت ۲ و ۲ ضرب

سِفْت ۳ و ۳ ضرب

۱۶ ضرب

نکته: در ضرب دو عدد m, n رقمی نیز به mn ضرب $O(n^2)$.

نکته: در ضرب دو عدد n, n رقمی نیز به n^2 ضرب $O(n^2)$.

ضرب اعداد بزرگ به روش تقسیم و حمله

مثال

$$x \begin{array}{|c|c|} \hline a & b \\ \hline \end{array}$$

$$y \begin{array}{|c|c|} \hline c & d \\ \hline \end{array}$$

$$ac \times 10^{n/2} + (ad + cb) \times 10^{n/2} + bd$$

تعداد ضرب مورد نیاز ۴

$$\begin{array}{|c|c|} \hline 2 & 1 \\ \hline \end{array}$$

$$\begin{array}{|c|c|} \hline 3 & 4 \\ \hline \end{array}$$

$$2 \times 3 \times 10^2 +$$

$$(2 \times 4 + 1 \times 3) \times 10 +$$

$$1 \times 4 = 714$$

تعداد ضرب ۴

اولین بار

$$x \begin{bmatrix} a & b \end{bmatrix}$$

$$y \begin{bmatrix} c & d \end{bmatrix}$$

$$u = ac$$

$$w = bd$$

$$r = (a+b)(c+d) - u - w$$

عمل ضرب

$$u \times 10^n + r \times 10^{n/2} + w =$$

$$ac \times 10^n + (ad + cb) \times 10^{n/2} + bd$$

مثال

$$\begin{bmatrix} 2 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 3 & 4 \end{bmatrix}$$

$$u = 2 \times 3 = 6$$

$$w = 1 \times 4 = 4$$

$$r = (2+1)(3+4) - 6 - 4 = 11$$

عمل ضرب

$$6 \times 10^2 + 11 \times 10 + 4 = 714$$

نکته: مسن این روش ظاهر ضرب دو عدد دورقمی نیز به کار برده می شود.

$$\begin{bmatrix} 5 & 6 \end{bmatrix}$$

$$\begin{bmatrix} 7 & 8 \end{bmatrix}$$

$$u = 5 \times 7 = 35$$

$$w = 6 \times 8 = 48$$

$$r = (5+6)(7+8) - 35 - 48 =$$

عمل ضرب

$$\begin{array}{r} 448 \\ 392 \\ \hline 4368 \end{array}$$

$$\begin{bmatrix} 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 5 \end{bmatrix}$$

$$u = 1 \times 1 = 1$$

$$w = 1 \times 5 = 5$$

$$r = (1+1)(1+5) - 1 - 5 = 6$$

تمرین: در ضرب دومر ۱۶ رقم به هم حاصل و حاصل ضرب چند عمل ضرب نیز است؟

در ضرب عادی $16 \times 16 = 256$ عمل ضرب نیز است

در روش بنی تسم و غنیه با ۳ حرفت (۳ رقم) به عمل ضرب

نیز است 16×16 تعداد ارقام 3×3 ضرب

8×8 تعداد ارقام 3×3 ضرب $4 = 8$ ضرب

4×4 3×3 ضرب

2×2 3×3 ضرب

نکته: در ضرب دومر n رقم به هم در ضرب عادی نیز است: n^2 g_2^n ضرب نیز است

نکته: در ضرب دومر n رقم به هم در ضرب بنی نیز است: n g_2^3 g_2^n ضرب نیز است

الگوریتم فشرده سازی هافمن (Huffman)

در روش های فشرده سازی، هر کاراکتر یک بیت (8 بیت) اختصاص داده می شود.

به این تبدیل ASCII می گویند: $'A' = 65 = 01000001$ (مثال)

$'D' = 68 = 01000100$

حال اگر بخواهیم داده های کاراکترها را فشرده کنیم، باید از هر کاراکتر 8 بیت اختصاص دهیم.

مثال: اگر بخواهیم داده های زیر را فشرده کنیم:

کاراکتر	A	B	C	D	E
افراد	11	7	3	8	6

$35 \times 8 = 280$ بیت
 $35 =$ تعداد کاراکتر

الگوریتم فشرده سازی هافمن

به کاراکترها، دیدن داده می شود. به کاراکترهایی که افراد بیشتری دارند، کد کوتاه تر و آنهایی که افراد کمتری دارند، کد بلندتر داده می شود.

روش کار:

جدول توزیع کارترها را از کم به زیاد مرتب می‌کنیم و دو کارتر اول را به لو بهترین
 فرادا را داریم انتخاب می‌کنیم و با آن یک دِف دو در می‌ایم (و بگتر است چ)
 پس این دو کارتر را از جدول حذف کرده حاصل جمع فرادا را به جدول اضافه می‌کنیم
 بعد جدول را مرتب می‌کنیم و دو کارتر اول به لو بهترین فرادا را داریم انتخاب

و دِف دو در می‌ایم.

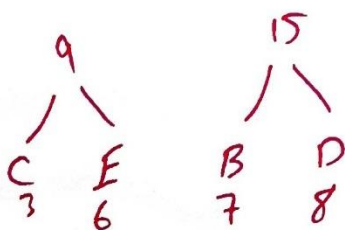
زمانی که همه کارترها انتخاب شدند یک دِف دو در می‌ایم و یک چ را

صفر و ۷ است است یک قرار می‌دهیم.

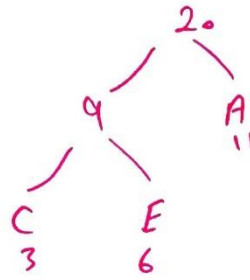
کارتر	A	D	B	E	C
فرادان	11	8	7	6	3



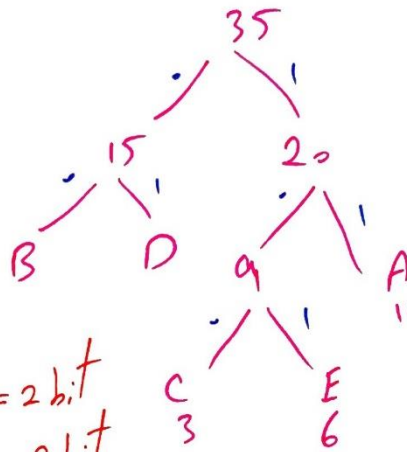
کارتر	A	CE	D	B
فرادان	11	9	8	7



کد باینری	CE	A	BD
فردان	9	11	15



کد باینری	BD	CEA
فردان	15	20



$$\begin{aligned}
 B &= 00 = 2 \text{ bit} \\
 D &= 01 = 2 \text{ bit} \\
 A &= 11 = 2 \text{ bit} \\
 C &= 100 = 3 \text{ bit} \\
 E &= 101 = 3 \text{ bit}
 \end{aligned}$$

کد باینری	A	B	C	D	E
فردان	11	7	3	8	6
تعداد بیت	2	2	3	2	3

$$11 \times 2 + 7 \times 2 + 3 \times 3 + 8 \times 2 + 6 \times 3 = 22 + 14 + 9 + 16 + 18 = 79 \text{ bit}$$

در ریش ۲۸ بیت

در ریش هامن ۷۹ بیت

تمرین: تعداد سبب مورثیه را در زیر جدول محاسبه کنید

و به روش هافمن چه سبب است؟

طایفه	A	B	C	D	E	F	G	H
فردان	22	5	11	19	2	11	25	5

طایفه	E	B	H	C	F	D	A	G
فردان	2	5	5	11	11	19	22	25

طایفه	H	EB	C	F	D	A	G
فردان	5	7	11	11	19	22	25

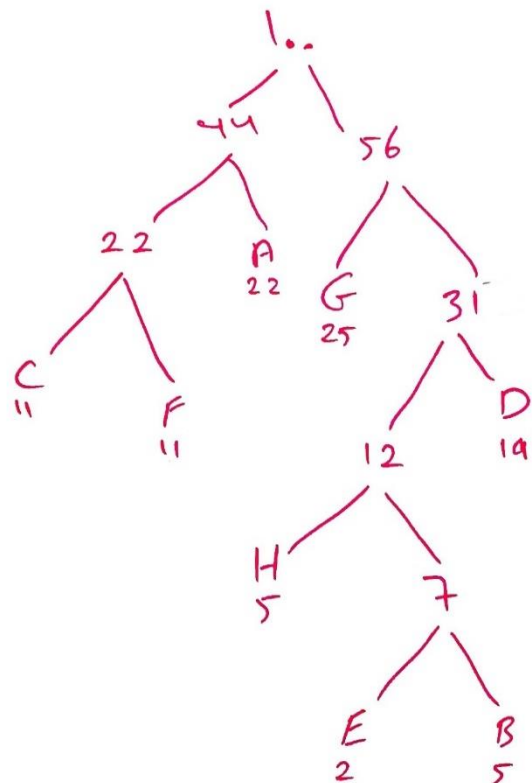
طایفه	C	F	HEB	D	A	G
فردان	11	11	12	19	22	25

طایفه	HEB	D	A	CF	G
فردان	12	19	22	22	25

طایفه	A	CF	G	HEBD
فردان	22	22	25	31

طایفه	G	HEBD	ACF
فردان	25	31	44

طایفه	ACF	GHEBD
فردان	44	56



وزارت علوم تحقیقات فناوری



دانشگاه فنی و حرفه ای

دانشکده شهید شمس پور تهران

درس طراحی الگوریتم

مقطع کارشناسی (نرم افزار)

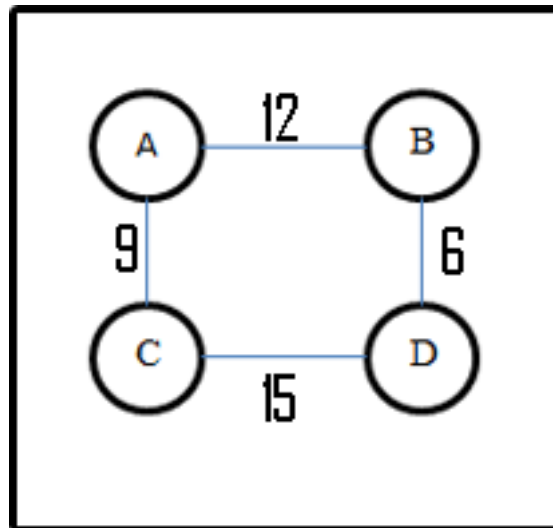
مدرس : دکتر عادل نیا

(الگوریتم حریصانه) Greedy algorithm

یک الگوریتم حریصانه، همانطور که از اسم آن مشخص است، حریص است و همیشه انتخابی که در آن لحظه بهترین به نظر می‌رسد را بر می‌گزیند. به این معنی که انتخاب را به گونه ای انجام می دهد که به صورت محلی و در همان موقعیت بهینه باشد، به امید آن که این انتخاب باعث شود جواب نهایی هم بهینه شود.

کجا نمیتوان از الگوریتم حریصانه استفاده کرد؟

گراف بدون جهت زیر را در نظر بگیرید.



برای پیدا کردن کوتاه ترین مسیر بین گره A و D اگر به صورت حریصانه عمل کنیم یال A-C از یال A-B دارای وزن کمتری میباشد پس یال A-C را در نظر میگیریم ولی در ادامه مجبوریم از تنها انتخاب یعنی C-D استفاده کنیم و به گره D برسیم. طول مسیر $9 + 15$ یعنی ۲۴ میباشد اما در صورتی که در ابتدا یال A-B را انتخاب کرده بودیم با اینکه نسبت به A-C در مرحله اول انتخاب بدتری بود اما در ادامه مسیر بهینه تر و کوتاه تری را داشتیم.

پس نمیتوان از الگوریتم حریصانه در پیدا کردن کوتاه ترین میسر بین دو گره استفاده کرد برای اینکار میتوان از الگوریتم هایی مثل Dijkstra استفاده کرد .

مزایای الگوریتم های حریصانه

- ✓ پیاده سازی الگوریتم حریصانه ساده است.
- ✓ الگوریتم حریصانه دارای پیچیدگی زمانی پایین است.
- ✓ از این الگوریتم می توان در مسائل پیچیده برای پیدا کردن مقدار تخمینی پاسخ به راحتی استفاده کرد.
- ✓ این الگوریتم به عنوان الگوریتمی کارآمد محسوب می شود، زیرا این الگوریتم تمامی راه حل های مسئله را بررسی نمی کند.
- ✓ درک این الگوریتم ساده است.

الگوریتم خرد کردن سکه

می خواهیم میزان پولی را پرداخت کنیم به نحوی که حداقل تعداد سکه پرداخت شود. برای مثال برای مجموعه سکه های $\{1, 5, 6\}$ و ۱۰ واحد پول می تواند جواب های مختلفی ارائه شود :

$$10 = \{6, 1, 1, 1, 1\} \text{ روش حریصانه}$$

$$10 = \{5, 1, 1, 1, 1\}$$

$$10 = \{5, 5\} \text{ روش بهینه}$$

برای مثال روش حریصانه برای مجموعه سکه های $\{1, 5, 6\}$ و ۱۰ واحد پول جواب غیر بهینه زیر را می دهد.

برای مثال روش حریصانه برای مجموعه سکه های $\{20, 10, 5, 1\}$ و ۴۶ واحد پول جواب بهینه زیر را می دهد.

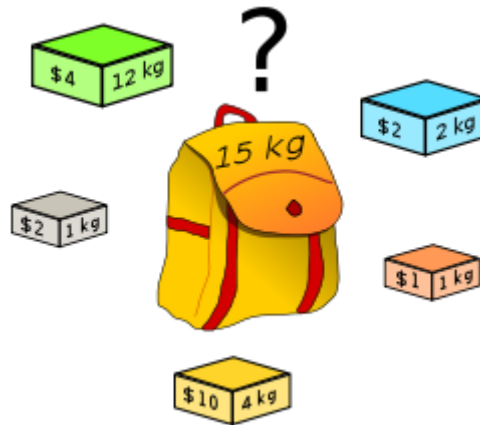
$$46 = \{20, 20, 5, 1\} \text{ روش حریصانه و بهینه}$$

$$46 = \{20, 10, 10, 5, 1\}$$

$$46 = \{10, 10, 10, 10, 5, 1\}$$

$$46 = \{20, 10, 5, 5, 5, 1\}$$

الگوریتم کوله پستی Knapsack Problem



A		B	
		Max : 15 kg	
	1.11 \$/kg	(1)	
	0.58 \$/kg	(2)	
	0.5 \$/kg	(3)	
	0.43 \$/kg	(4)	
	0.4 \$/kg	(5)	



Wt. = 5
 Value = 10



Wt. = 3
 Value = 20



Wt. = 8
 Value = 25



Wt. = 4
 Value = 8



Maximum wt. = 13

کدومو داخل کوله پستی بزارم که سودم بیشتر بشود؟



مرتبۀ اجرایی الگوریتم های میانگین ، مد ، میانۀ

الگوریتم حاصل جمع دارای مرتبۀ $O(n)$

2, 3, 4

$$2 + 3 + 4 = 9$$

الگوریتم میانگین دارای مرتبۀ $O(n)$

2, 3, 4

1 2 ③

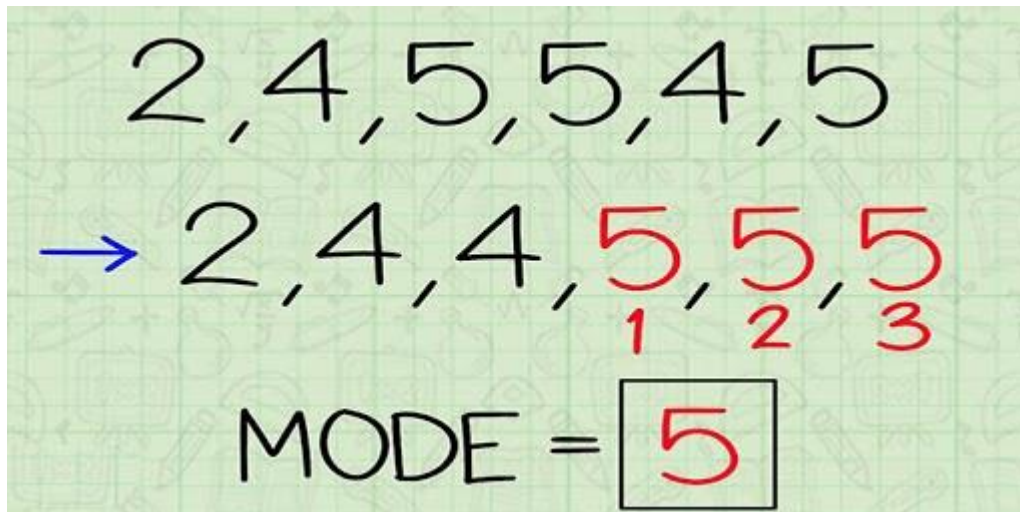
$$2 + 3 + 4 = 9$$

$$\text{MEAN} = 9 \div 3 = \boxed{3}$$

الگوریتم پیدا کردن مد دارای مرتبه $O(n^2)$

با استفاده از الگوریتم مرتب سازی مرتبه $O(n \log n)$

با استفاده از آرایه بزرگ و الگوریتم خاص مرتبه $O(n)$



برای یافتن مد، ابتدا اعداد را مرتب می کنیم، سپس تعداد هر عدد را بدست می آوریم. عددی که بیشتر از همه موجود باشد، مد است.

مثال:

۳, ۷, ۵, ۱۳, ۲۰, ۲۳, ۳۹, ۲۳, ۴۰, ۲۳, ۱۴, ۱۲, ۵۶, ۲۳, ۲۹

اگر اعداد ذکر شده را مرتب کنیم، به ترتیب زیر خواهند بود:

۳, ۵, ۷, ۱۲, ۱۳, ۱۴, ۲۰, ۲۳, ۲۳, ۲۳, ۲۳, ۲۹, ۳۹, ۴۰, ۵۶

مرتب کردن اعداد باعث می شود تعداد هر عدد را براحتی بدست آوریم

الگوریتم پیدا کردن می نیم نیاز به مرتبه $O(n)$

0	1	2	3	4	5	6	7	8	9
35	33	42	10	14	19	26	44	26	31

0	1	2	3	4	5	6	7	8	9
10	14	19	26	26	31	33	35	42	44

الگوریتم پیدا کردن K امین مینیم نیاز به مرتبه $O(n)$ دارد**پیدا کردن هفتمین کوچکترین عنصر**

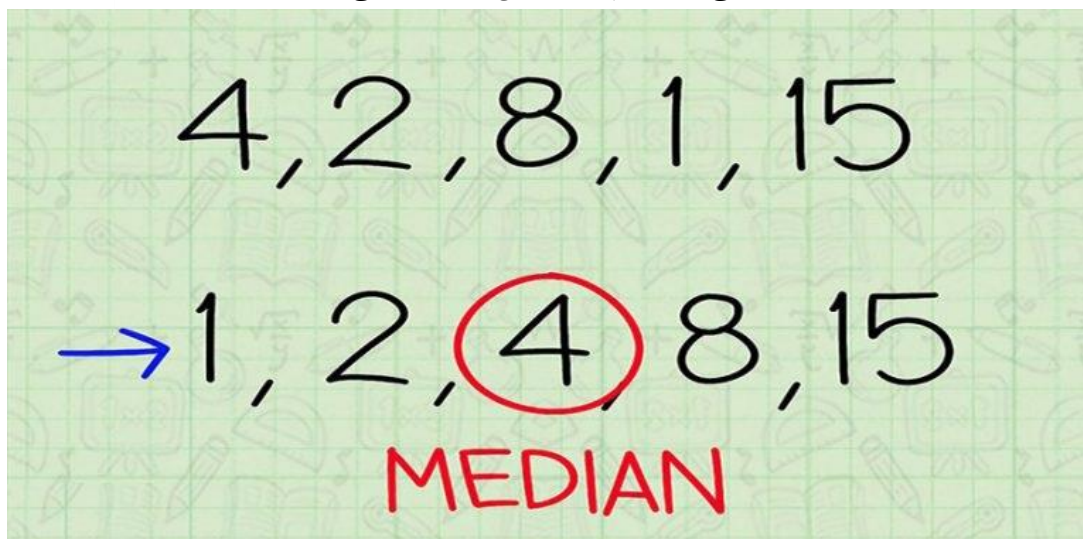
6	10	13	5	8	3	2	11	$i = 7$	$(2,3,5,6,8,10,11,13)$
---	----	----	---	---	---	---	----	---------	------------------------

عنصر اول را محور در نظر می گیریم و پارتیشن بندی می کنیم:

2	5	3	6	8	13	10	11	$k = 4$
---	---	---	---	---	----	----	----	---------

به طور بازگشتی در قسمت صورتی رنگ به دنبال سومین کوچکترین عنصر می گردیم.

الگوریتم پیدا کردن میانه دارای مرتبه $O(n)$



اگر با روش مرتب سازی عمل کنیم پیدا کردن میانه نیاز به مرتبه $O(n \log n)$ دارد
اگر با روش پارتیشن بندی که در روش مرتب سازی سریع مطرح می باشد عمل کنیم نیاز به مرتبه $O(n)$ می باشد



الگوریتمهای مرتب سازی آرایه ها

Array Sorting Algorithms

ساختمان داده ها و الگوریتم ها

فهرست

جدول مقایسه ی الگوریتم ها

نمودارهای مقایسه ی الگوریتم ها

الگوریتم ها:

۱- مرتب سازی حبابی (Bubble)

۲- مرتب سازی درجی (Insertion)

۳- مرتب سازی انتخابی (Selection)

۴- مرتب سازی شل (Shell)

۵- مرتب سازی ادغامی (Merge)

۶- مرتب سازی هرمی (Heap)

۷- مرتب سازی سریع (Quick)



الگوریتم	بهترین B(n)	میانگین A(n)	بدترین W(n)	حافظه*	پایدار	روش عمومی
حبابی Bubble	n	n^2	n^2	1	بله	جابجایی
درجی Insertion	n	n^2	n^2	1	بله	درجی
انتخابی Selection	n^2	n^2	n^2	1	خیر	انتخابی
شل Shell	n	$n \log^2 n$	$n \log^2 n$	1	خیر	درجی
ادغامی Merge	nlogn	nlogn	nlogn	بستگی* دارد	بله	ادغامی
هرمی Heap	nlogn	nlogn	nlogn	1	خیر	انتخابی
سریع Quick	nlogn	nlogn	n^2	Log n	بستگی دارد	جزءبندی

*حافظه : بیانگر مقدار حافظه کمکی که علاوه بر حافظه اشغال شده توسط خود داده ها، مورد استفاده الگوریتم قرار میگیرد.

* پایدار بودن آن به پیاده سازی بستگی دارد.



1- مرتب سازی حبابی (Bubble)

ساده ترین روش مرتب سازی الگوریتم ها به شمار می آید. در این روش هر عنصر با عنصر بعدی اش مقایسه میشود. در صورتی که عنصر دومی کوچکتر از عنصر اولی باشد، جای دو عنصر با هم عوض میشود. برنامه به کارش ادامه میدهد و عناصر دوم و سوم را با هم مقایسه میکند و این کار را تا آخر آرایه ادامه میدهد. دوباره الگوریتم ، پویش را از اول آرایه شروع میکند و مراحل قبل را تکرار میکند و این مراحل آنقدر تکرار میشوند تا آرایه کاملاً مرتب شده باشد، در نتیجه بازده پایین می آید.

مرتبه ی پیچیدگی این الگوریتم $O(n^2)$ است.

*یکی از معایب این الگوریتم ، تعداد مقایسه های زیاد آن میباشد و اگر آرایه، عناصر زیادی داشته باشد، وقت زیادی را برای مرتب کردن عناصر از برنامه و CPU خواهد گرفت.



مرحله اول

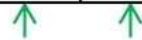
5	1	12	-3	11
---	---	----	----	----



1	5	12	-3	11
---	---	----	----	----



1	5	-3	12	11
---	---	----	----	----



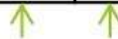
1	5	-3	11	12
---	---	----	----	----

مرحله دوم

1	5	-3	11	12
---	---	----	----	----

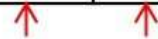


1	-3	5	11	12
---	----	---	----	----

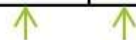


مرحله سوم

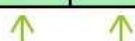
1	-3	5	11	12
---	----	---	----	----



-3	1	5	11	12
----	---	---	----	----



-3	1	5	11	12
----	---	---	----	----



کد الگوریتم Bubble Sort به زبان C++

```
void bubble (int x[],int n)
{
int temp;
for ( i=n-1;i>0;i-- )
    for ( j=0;j<i;j++ )
        if (x[j] > x[j+1])
        {
            temp = x[j];
            x[j] = x[j+1];
            x[j+1] = temp;
        }
}
```

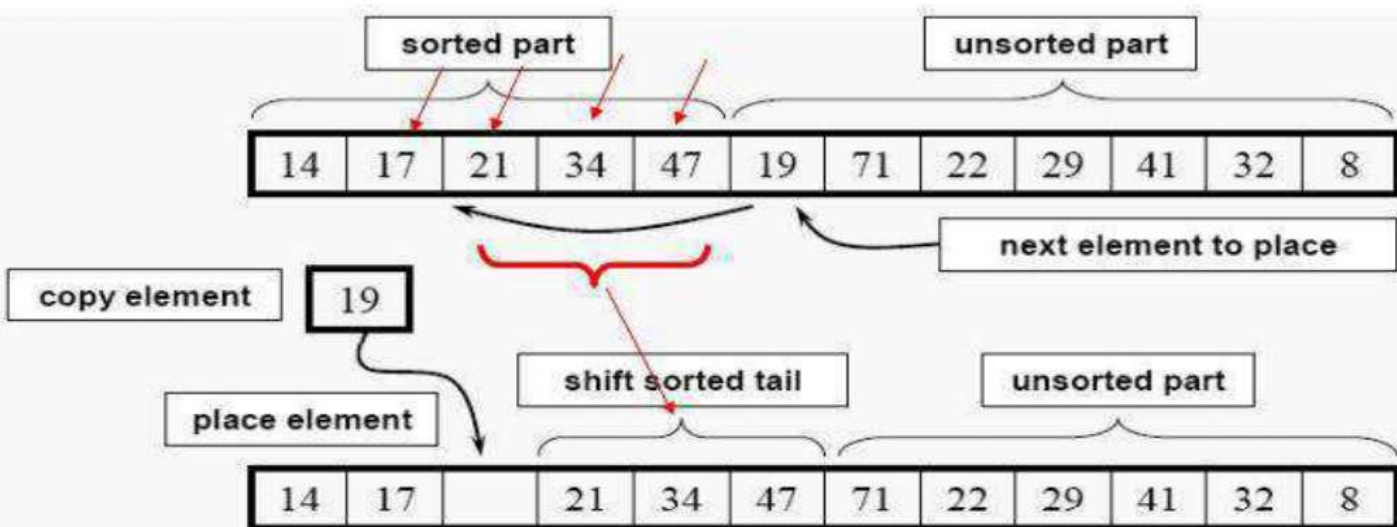
Bubble Sort with Flag

```
int swap = 1 ;
for (int pass = 1; pass < .n && swap == 1; pass++) {
    swap = 0;
    for (int j = 0; j < .n - pass; j++){
        if ( A[j] > A[j+1]) {
            int temp = A[j];
            A[j] = A[j+1];
            A[j+1] = temp;
            swap = 1;
        }
    }
}
```


2- مرتب سازی درجی (Insertion)

این الگوریتم نیز تقریباً مانند الگوریتم حبابی عمل میکند با این تفاوت که در این روش مقایسه از عنصر دوم شروع میشود، این الگوریتم عنصر اول و دوم را با هم مقایسه میکند و در صورت نیاز مرتب میشوند و سپس سومین عنصر با عناصر اول و دوم مقایسه میشود. در صورتی که عنصر سوم از اولی کوچکتر باشد به جای اولین عنصر می نشیند و عناصر قبلی به سمت راست هل داده میشوند. اگر عنصر سوم از اولی بزرگتر و از دومی کوچکتر باشد، بین آنها درج میشود و عنصر دوم به بعد یکی به سمت راست هل داده میشود. (پس در این روش همیشه عناصر ، قبل از عنصری که میخواهیم مرتبش کنیم، مرتب هستند.) این روال برای بقیه عناصر نیز اجرا میشود و هر عنصر در جای خودش قرار می گیرد تا تمام عناصر مرتب شوند.

در تکرار i ام این الگوریتم ، i عنصر اول در آرایه اصلی مرتب خواهند شد. مرتبه ی پیچیدگی این الگوریتم $O(n^2)$ است.



5 1 12 -3 14

1 5 12 -3 14

1 5 12 -3 14

1 5 12 14

temp

-3

-3 1 5 12 14

-3 1 5 12 14

-3 1 5 12 14

Pass: 1

4 3 1 2 0

Insert A[1] in left sorted list

3 4 1 2 0
sorted unsorted

Pass: 2

3 4 1 2 0

Insert A[2] in left sorted list

1 3 4 2 0
sorted unsorted

Pass: 3

1 3 4 2 0

Insert A[3] in left sorted list

1 2 3 4 0
sorted unsorted

Pass: 4

1 2 3 4 0

Insert A[4] in left sorted list

0 1 2 3 4
sorted

کد الگوریتم Insertion Sort به زبان C++

```
void insertion (int x[],int n)
{
    int temp;
    for( i=1;i<n;i++)
    {
        j=i;
        while( j>0 && x[j-1]>x[j])
        {
            temp = x[j];
            x[j] = x[j-1];
            x[j-1] = temp;
            j--;
        }
    }
}
```

```
void insertion (int x[],int n)
{
    int temp;
    for( i=1;i<n;i++)
    {
        for( j=i; j>0 && x[j-1]>x[j]; j-- )
        {
            temp = x[j];
            x[j] = x[j-1];
            x[j-1] = temp;
        }
    }
}
```

3- مرتب سازی انتخابی (Selection)

در این روش، برنامه کوچکترین مقدار را یافته و آنرا در اولین خانه ی آرایه قرار می دهد. حال که کوچکترین عضو یافت شده است، برنامه به سراغ یافتن دومین عنصر کوچک در میان اعداد باقی مانده که از 2 تا n هستند می رود و دومین عدد کوچک را در خانه دوم قرار میدهد. حال به سراغ سومین عدد کوچک می رود و این رویه را تا یافتن آخرین عدد و قرار دادن آن در جای خودش تکرار میکند. با توجه به اینکه برنامه باید n عدد را n بار با هم مقایسه کند مرتبه ی پیچیدگی این الگوریتم $O(n^2)$ است.

*الگوریتم انتخابی 60% سریعتر از الگوریتم حبابی است، اما الگوریتم درجی بهتر از انتخابی و حبابی اجرا می شود.

*در الگوریتم درجی برای پیدا کردن عنصر x هر تعداد از اعداد را که نیاز است بررسی می کند ولی الگوریتم انتخابی همه عناصر باقیمانده را بررسی میکند.



5	1	12	-3	14
---	---	----	----	----

↑ ↑

-3	1	12	5	14
----	---	----	---	----

↑

-3	1	12	5	14
----	---	----	---	----

↑ ↑

-3	1	5	12	14
----	---	---	----	----

↑

-3	1	5	12	14
----	---	---	----	----

↑

3	1	5	12	14
---	---	---	----	----

کد الگوریتم Selection Sort به زبان C++

```
void selection sort (int x[], int n)
{
    int i , j , item , min;
    for (i=0 ; i<n-1 ; i++)
    {
        min = i ;
        for ( j=i+1 ; j<n ; j++ )
            if (x[j] < x[min])
                min = j ;
        item = x[i] ;
        x [i] = x[min] ;
        x[min] = item ;
    }
}
```

4- مرتب سازی شل (Shell)

این روش یکی از روش های مرتب سازی درجی است که بهبود یافته روش درج ساده عمل می باشد و اولین بار توسط Shell طراحی گردید.

در این روش که در چند گذر انجام می شود، در هر گذر لیست موجود، به مجموعه های مجزایی از عناصر افزای می شود و هر کدام از این قسمت ها معمولاً به روش درج ساده مرتب می شوند.

در هر گذر برای تقسیم بندی لیست به مجموعه هایی مجزا از پارامتر مشخصی مانند k استفاده می شود که پارامتر «افزایش» نامیده می شود.

به عنوان مثال اگر $k=3$ انتخاب شود، لیست مورد نظر به 3 مجموعه مجزا به صورت زیر تقسیم می شود:

$$S1 = \{ a[0], a[3], a[6], \dots \}$$

$$S2 = \{ a[1], a[4], a[7], \dots \}$$

$$S3 = \{ a[2], a[5], a[8], \dots \}$$

برای اینکه عنصر i ام از $s(j)$ یک آرایه را بدست آوریم از فرمول زیر استفاده می کنیم:

$$a[(i-1).k+j-1]$$

به عنوان مثال عنصر دوم از $S3$ برابر است با

$$a[(2-1)*3+3-1] = a[5]$$

*روش درج ساده روشی است که در آن هرچه لیست اولیه مرتب تر باشد تعداد عملیات لازم جهت مرتب سازی کمتر است.

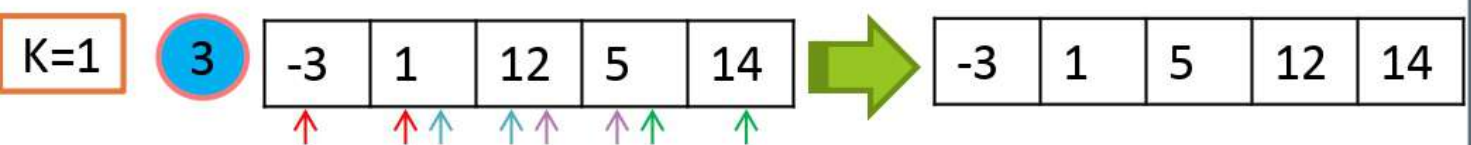
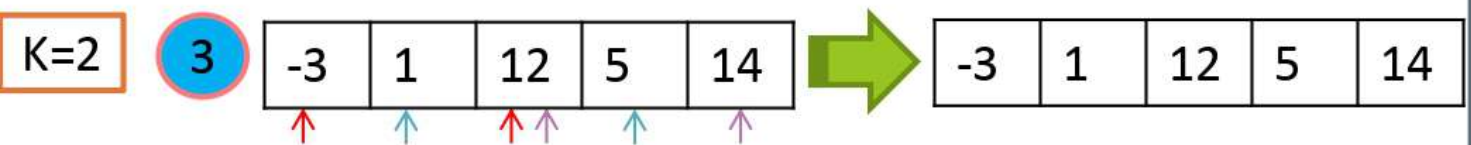
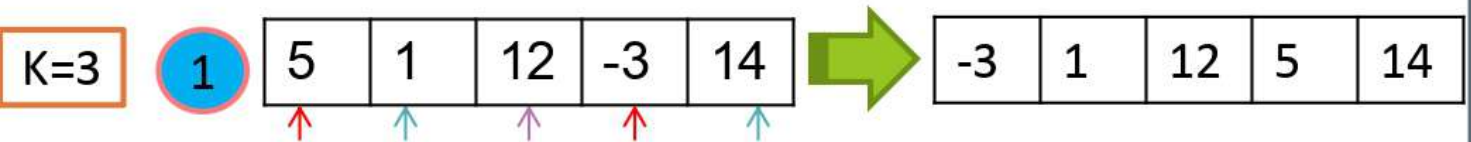
*از طرف دیگر هرچه تعداد عناصر لیست بیشتر باشد تعداد عملیات بیشتری نیاز خواهد بود.

*روش مرتب سازی شل با در نظر گرفتن این نکات سعی دارد تا اولاً روش درج ساده را با قسمت بندی عناصر به مجموعه هایی مجزا بر روی مجموعه هایی با طول کوچکتر انجام دهد. ثانیاً در هر گذر، لیست اولیه را برای مراحل بعدی مرتب تر نماید.



5	1	12	-3	14
---	---	----	----	----

مرحله فاصله



-3	1	5	12	14
----	---	---	----	----

کد الگوریتم Shell Sort به زبان C++

```
void shellsort(numbers[size])
{
    int i,j,increment,temp;
    increment=3;
    while(increment>0)
    {
        for(i=0;i<size;i++)
        {
            j=i;
            temp=numbers[i];
            while((j>=increment) && (numbers[j-increment]>temp))
            {
                numbers[j]=numbers[j-increment];
                j=j-increment;
            }
            numbers[j]=temp;
        }
        if(increment/2 !=0)
            increment=increment/2;
        else if(increment==1)
            increment=0;
        else
            increment=1;
    }
}
```


5- مرتب سازی ادغامی (Merge Sort)

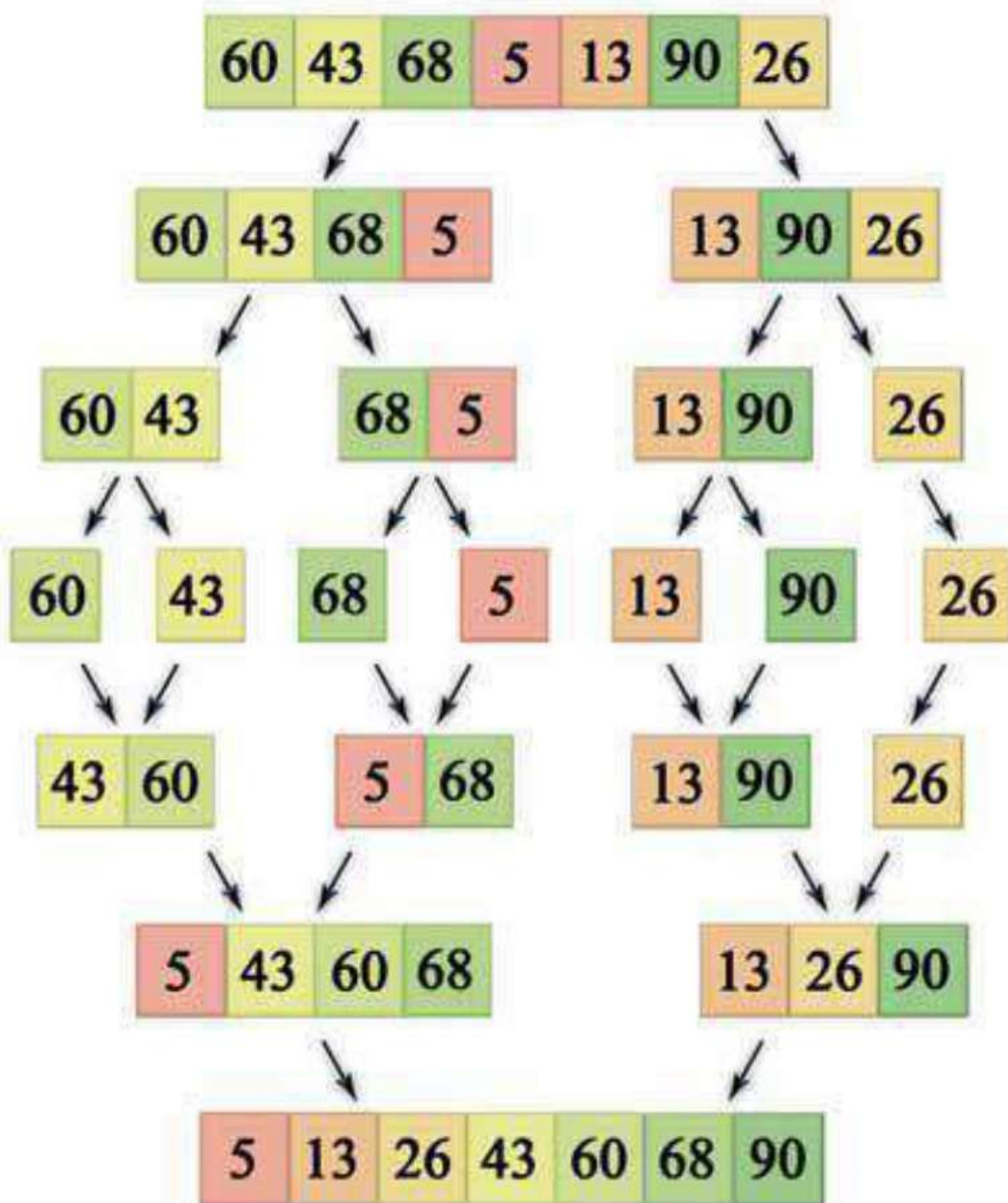
این الگوریتم به روش بازگشتی (Recursive) عمل میکند و آرایه را به چند آرایه ی دو عنصری تقسیم میکند و آنها را مرتب میکند. سپس آرایه های کوچک را دوبه دو با هم ادغام میکند تا آرایه های مرتب 4 عنصری ایجاد شوند و بعد آرایه های 8 عنصری و به همین ترتیب پیش می رود تا آرایه اصلی بصورت مرتب شده ظاهر شود.

این روش از مراحل بازگشتی زیر تشکیل یافته است:

- 1- آرایه را به دو زیر آرایه با اندازه تقریباً یکسان تقسیم کن.
- 2- دو زیر آرایه را به روش مرتب سازی ادغامی مرتب کن.
- 3- دو زیر آرایه مرتب شده را ادغام کن.

مرتبه پیچیدگی این الگوریتم $O(n \log n)$ است.

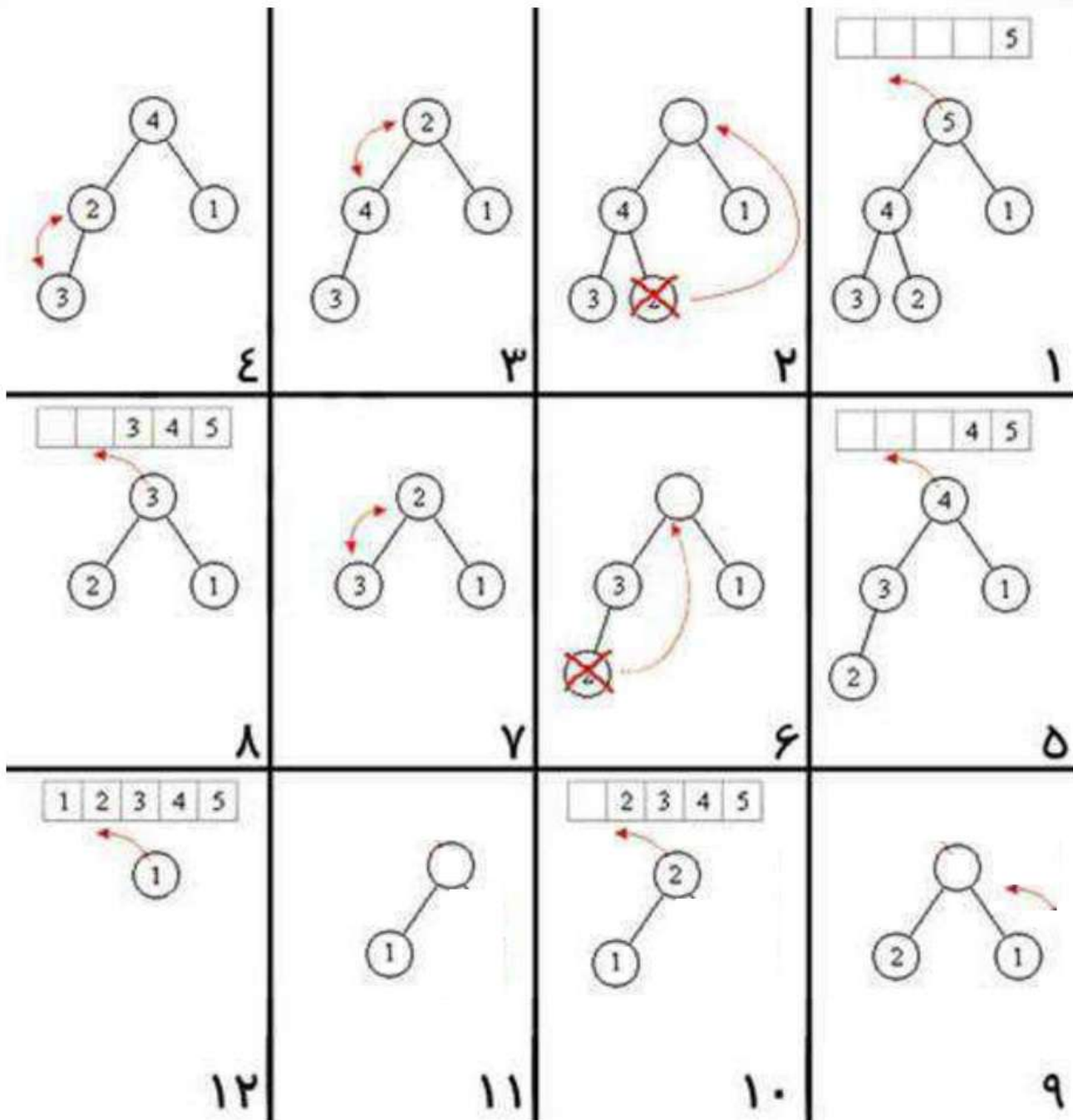




6- مرتب سازی هرمی (Heap Sort) :

یک الگوریتم کارای مرتب سازی درجا در حالت کلی است که در بدتری حالت پیچیدگی زمانی بهینه دارد، هرچند که در عمل، مرتب سازی سریع از آن سریعتر عمل میکند. این الگوریتم بر اساس داده ساختارهای هرم است. در این روش، برنامه از کل آرایه ی داده شده یک درخت MaxHeap می سازد. (درخت مکس هیپ درختی دودویی و کامل است که مقدار ذخیره شده در هر گره، بزرگتر و یا مساوی مقدار ذخیره شده در گره فرزندانش است) سپس مقدار ماکزیمم را از درخت حذف میکند و آنرا در انتهای آرایه میگذارد و دوباره از بقیه اعداد یک درخت maxHeap میسازد و باز روش مذکور را روی آن نیز اعمال میکند تا دومین عدد بزرگ یافت شود. این کار تا زمانی که هیچ مقداری در Heap نماند و آرایه مرتب شده کامل شود، تکرار خواهد شد. در این روش آرایه از آخر به اول مرتب میشود. مرتبه پیچیدگی این الگوریتم $O(n \log n)$ است.





7- مرتب سازی سریع (Quick):

در این الگوریتم یک عنصر را بعنوان محور (pivot) مرتب سازی انتخاب میکنیم. و تمام عناصر کوچکتر از آن را به سمت چپ آن برده و عناصر بزرگتر را به سمت راست اش می‌بریم. حالا بخش چپ خودش یک بخش جدید است که با الگوریتمی که گفتیم آنرا مرتب میکنیم و سمت راست را نیز همینطور. یعنی در سمت چپی ها دوباره یک عنصر را بعنوان pivot در نظر میگیریم و عناصر کوچکتر از pivot را به سمت چپ آن و عناصر بزرگتر از pivot این قسمت را ، به سمت راست pivot می‌بریم. دوباره الگوریتم را روی یک چهارم های به وجود آمده اجرا میکنیم و اینکار را آنقدر ادامه میدهیم تا کل آرایه مرتب شود.

مرتبه پیچیدگی این الگوریتم در بدترین حالت $O(n^2)$ است. اما در حالت میانگین $O(n \log n)$ است که کمترین مرتبه پیچیدگی برای مرتب سازی اعداد به حساب می‌آید.

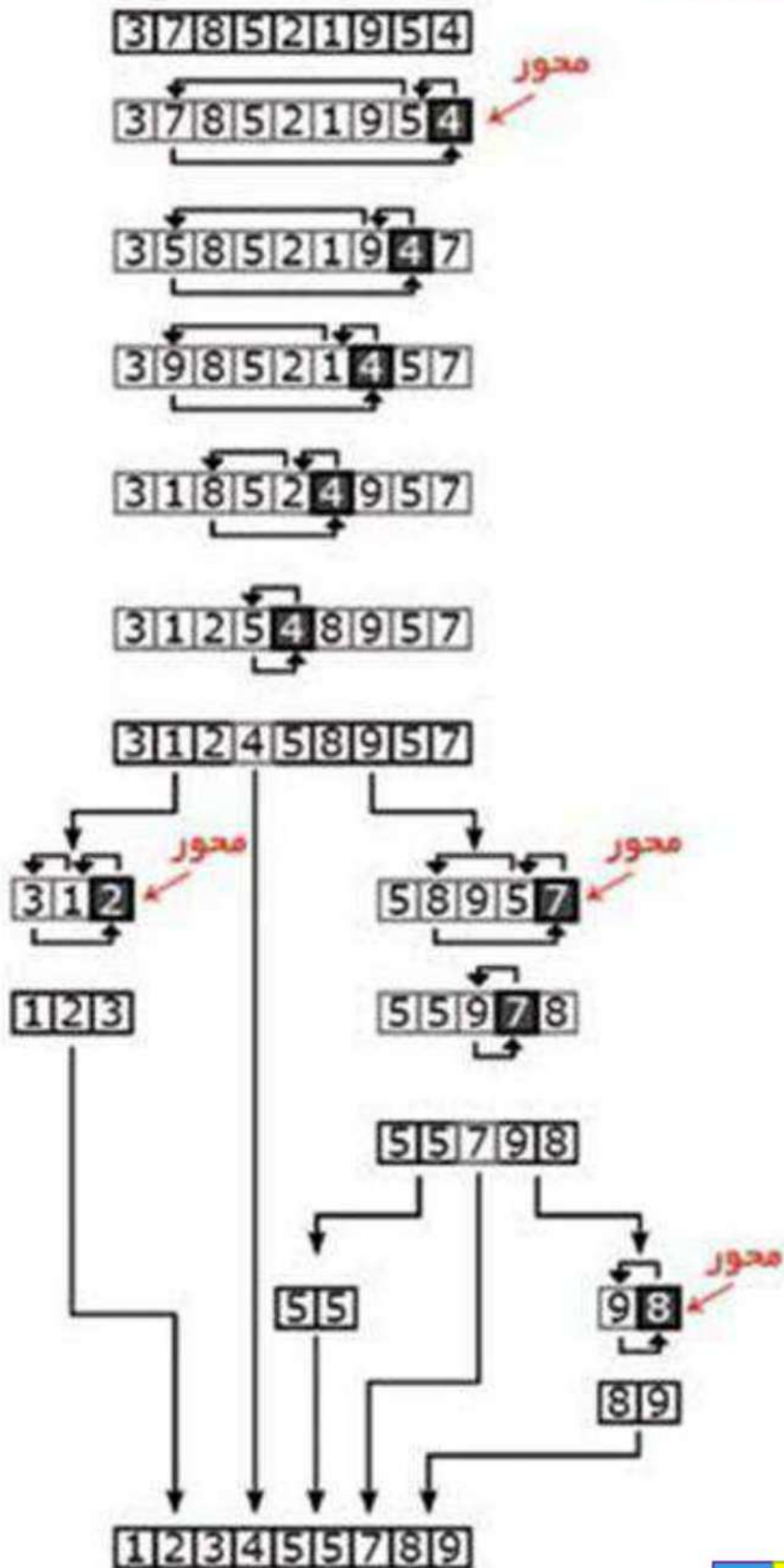
* در عمل سریعتر از الگوریتم های بهینه دیگر مرتب میکند.

* گونه ی حافظه خارجی این الگوریتم کاراست.

* پیاده سازی آن به صورت موازی هم امکان پذیر است.



تحليل الگوریتم :



کد الگوریتم Quick Sort به زبان C++

```
void quicksort(int x[],int left,int right)
{
    int i=left,j=right;
    int temp;
    int pivot=x[(left+right)/2];
    while (i<=j)
    {
        while(x[i]<pivot)
            i++;
        while(x[j]>pivot)
            j--;
        if(i<=j)
        {
            temp=x[i];
            x[i]=x[j];
            x[j]=temp;
            i++;
            j--;
        }
    }
    if (left<j)
        quicksort(x,left,j);
    if (i<right)
        quicksort(x,i,right);
}
```



وزارت علوم تحقیقات فناوری

دانشگاه فنی و حرفه ای

دانشکده شهید شمسی پور تهران

درس طراحی الگوریتم

مقطع کارشناسی (نرم افزار - شبکه)

مدرس : دکتر عادل نیا

مرتب سازی مبنایی Radix Sort

مثال اول

Sort Digit 0	Sort Digit 1	Sort Digit 2	Final Result
9 5 4	4 1 1	0 0 9	0 0 9
3 5 4	9 5 4	4 1 1	3 5 4
0 0 9	3 5 4	9 5 4	4 1 1
4 1 1	0 0 9	3 5 4	9 5 4

مثال دوم

329	720	720	329
457	355	329	355
657	436	436	436
839	457	839	457
436	657	355	657
720	329	457	720
355	839	657	839

مثال سوم

3 2 9	7 2 0	7 2 0	3 2 9
4 5 7	3 5 5	3 2 9	3 5 5
6 5 7	4 3 6	4 3 6	4 3 6
8 3 9	4 5 7	8 3 9	4 5 7
4 3 6	6 5 7	3 5 5	6 5 7
7 2 0	3 2 9	4 5 7	7 2 0
3 5 5	8 3 9	6 5 7	8 3 9
Sort	sorted	sorted	sorted

Consider the following list of unsorted integer numbers

82, 901, 100, 12, 150, 77, 55 & 23

Step 1 - Define 10 queues each represents a bucket for digits from 0 to 9.



Step 2 - Insert all the numbers of the list into respective queue based on the Least significant digit (once placed digit) of every number.

82, 901, 100, 12, 150, 77, 55 & 23



Group all the numbers from queue-0 to queue-9 in the order they have inserted & consider the list for next step as input list.

100, 150, 901, 82, 12, 23, 55 & 77

Step 3 - Insert all the numbers of the list into respective queue based on the next Least significant digit (Tens placed digit) of every number.

100, 150, 901, 82, 12, 23, 55 & 77



Group all the numbers from queue-0 to queue-9 in the order they have inserted & consider the list for next step as input list.

100, 901, 12, 23, 150, 55, 77 & 82

Step 4 - Insert all the numbers of the list into respective queue based on the next Least significant digit (Hundred placed digit) of every number.

100, 901, 12, 23, 150, 55, 77 & 82



Group all the numbers from queue-0 to queue-9 in the order they have inserted & consider the list for next step as input list.

12, 23, 55, 77, 82, 100, 150, 901

List got sorted in the increasing order.

Complexity of the Radix Sort Algorithm

To sort an unsorted list with 'n' number of elements, Radix sort algorithm needs the following complexities...

Worst Case : $O(n)$

Best Case : $O(n)$

Average Case : $O(n)$

Implementation of Radix Sort Algorithm using C Programming Language

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>

int getMax(int list[], int n) {
    int mx = list[0];
    int i;
    for (i = 1; i < n; i++)
        if (list[i] > mx)
            mx = list[i];
    return mx;
}

void countSort(int list[], int n, int exp) {
    int output[n];
    int i, count[10] = { 0 };

    for (i = 0; i < n; i++)
        count[(list[i] / exp) % 10]++;

    for (i = 1; i < 10; i++)
        count[i] += count[i - 1];

    for (i = n - 1; i >= 0; i--) {
        output[count[(list[i] / exp) % 10] - 1] = list[i];
        count[(list[i] / exp) % 10]--;
    }

    for (i = 0; i < n; i++)
        list[i] = output[i];
}

void radixsort(int list[], int n) {
    int m = getMax(list, n);

    int exp;
    for (exp = 1; m / exp > 0; exp *= 10)
        countSort(list, n, exp);
}
```

```
void print(int list[], int n) {
    int i;
    for (i = 0; i < n; i++)
        printf("%d\t", list[i]);
}

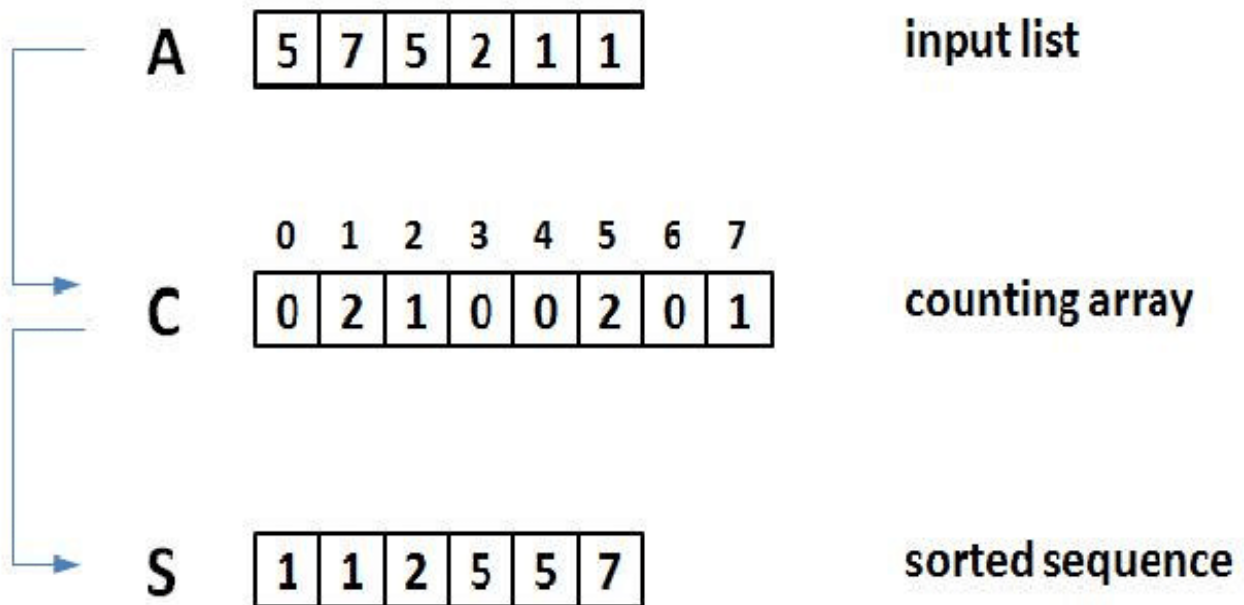
int main()
{
    int list[] = { 82, 901, 100, 12, 150, 77, 55, 23 };
    int i, n = sizeof(list) / sizeof(list[0]);

    printf("List of numbers before sort: \n");
    for(i = 0; i<8; i++)
        printf("%d\t", list[i] );

    radixsort(list, n);

    printf("\n\nList of numbers after sort: \n");
    print(list, n);
    printf("\n\n");
    return 0;
}
```


مرتب سازی شمارشی Counting Sort



Counting sort – pseudocode

```

COUNTING-SORT( $A, B, n, k$ )
  let  $C[0..k]$  be a new array
  for  $i = 0$  to  $k$ 
     $C[i] = 0$ 
  for  $j = 1$  to  $n$ 
     $C[A[j]] = C[A[j]] + 1$ 
  for  $i = 1$  to  $k$ 
     $C[i] = C[i] + C[i - 1]$ 
  for  $j = n$  downto 1
     $B[C[A[j]]] = A[j]$ 
     $C[A[j]] = C[A[j]] - 1$ 
  
```

- B is the preallocated output array

÷

Array Sorting Algorithms

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
Quicksort	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n^2)$	$O(\log(n))$
Mergesort	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$
Timsort	$\Omega(n)$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$
Heapsort	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(1)$
Bubble Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$\Omega(n^2)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
Tree Sort	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n^2)$	$O(n)$
Shell Sort	$\Omega(n \log(n))$	$\Theta(n(\log(n))^2)$	$O(n(\log(n))^2)$	$O(1)$
Bucket Sort	$\Omega(n+k)$	$\Theta(n+k)$	$O(n^2)$	$O(n)$
Radix Sort	$\Omega(nk)$	$\Theta(nk)$	$O(nk)$	$O(n+k)$
Counting Sort	$\Omega(n+k)$	$\Theta(n+k)$	$O(n+k)$	$O(k)$
Cubesort	$\Omega(n)$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$

Common Data Structure Operations

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
<u>Array</u>	$\theta(1)$	$\theta(n)$	$\theta(n)$	$\theta(n)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<u>Stack</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
<u>Queue</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
<u>Singly-Linked List</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
<u>Doubly-Linked List</u>	$\theta(n)$	$\theta(n)$	$\theta(1)$	$\theta(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
<u>Skip List</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n \log(n))$
<u>Hash Table</u>	N/A	$\theta(1)$	$\theta(1)$	$\theta(1)$	N/A	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<u>Binary Search Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<u>Cartesian Tree</u>	N/A	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	N/A	$O(n)$	$O(n)$	$O(n)$	$O(n)$
<u>B-Tree</u>	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$

زمانه‌ی با اهمیت معنی

کارها یک هدف و یک بهره معنی دارند. اگر بخواهیم آن‌ها را به هدف تبدیل کنیم، باید آن‌ها را به گونه‌ای انجام دهیم که به هدف منتهی شود.

هدف آنست که کارها به نحوی زمانه‌ی معنی شوند به بهترین بهره به دست آورده و این کار را از آن جهت که کارها زمانه‌ی معنی شوند.

ابتدا کارها را به یک بهره به صورت سری مرتب می‌کنیم و مجموعه S را ابتدا می‌کنیم. اگر می‌توانیم، یک کار به بهترین بهره را انتخاب کرده و وقتاً به مجموعه S اضافه می‌کنیم. اگر این مجموعه S اکنون به آن کار را جزو S قرار می‌دهیم و به آن اضافه می‌کنیم.

کار	J1	J2	J3	J4	بهره سری	کار	J1	J2	J3	J4	بهره سری
سود	30	35	25	40	→	سود	40	35	30	25	→
اهمیت	2	1	2	1	بهره سری سود	اهمیت	1	1	2	2	بهره سری سود

کار J4 را به عنوان انتخاب کرده، J2 را از فهرست خارج می‌کنیم. J1 را به عنوان J3 و J4 را به عنوان J2 می‌بینیم. لذا راه حل بهینه J1، J4، J3 با سود 70.

کار / عمل	J1	J2	J3	J4	J5	J6	J7
سود	40	35	30	25	20	15	10
مهلت	3	1	1	3	1	3	2

$[J1, J2, J4] \xrightarrow{\text{با دینار}} [J2, J1, J4]$

40 35 25

با سود 100

تاریخ 89:1

هویت از کارهای زیر دینار و امتداد حق اجرا. هویت از این کارها را
 نوع فاصله Deadline و این نوع به بعد از نوع فاصله انجام شود
 جریمه (Penalty) فاصله از این کارها برای اجرا بهترین جریمه
 زمانی نسبت به تمام موارد جریمه مقدار؟

work	w_1	w_2	w_3	w_4	w_5	w_6	w_7
Deadline	7	2	3	3	2	5	1
Penalty	10	40	50	60	70	30	20

ابتدا کارها را برای جریمه از بزرگ به کوچک مرتب کنیم

work	w_5	w_4	w_3	w_2	w_6	w_7	w_1
Deadline	2	3	3	2	5	1	7
Penalty	7.	6.	5.	4.	3.	2.	1.

$[5] \rightarrow [5, 4] \rightarrow [5, 4, 3] \rightarrow [5, 4, 3, \cancel{2}] \rightarrow [5, 4, 3, 6] \rightarrow$

$[5, 4, 3, 6, \cancel{7}] \rightarrow [5, 4, 3, 6, 1]$

چون کارهای 2, 7 اجرا نشدند، هزینه قانونی جریمه $4.0 + 2.0 + 6.0 = 12.0$

نتیجه

در جدول زیر مجریه از کارها به همراه مهلت، سود و درآمد را مشاهده می‌کنیم.
 یک طرفه در صورتی سود آن، دریافت می‌شود که آن کار بعد از مهلت
 تعریف انجام شده باشد. فرض کنید که کار در زمان انجام می‌شود. حداکثر سود حاصل
 از اجرا برآ مجریه فوق‌الحد است.

	J_1	J_2	J_3	J_4	J_5	J_6	J_7
سود	4.0	15	6.0	2.	1.	45	55
مهلت	2	4	3	2	3	2	1

6, 6	73	77	76	71	74	72	75
عدد	60	55	45	40	20	15	10
حالت	3	1	2	2	2	4	3

کد ها را براساس سورت از بزرگ به کوچک مرتب کنیم

$[3] \rightarrow [7, 3] \rightarrow [7, 6, 3] \rightarrow [7, 6, 3, \cancel{4}] \rightarrow$

$[7, 6, 3, \cancel{4}] \rightarrow [7, 6, 3, 2] \rightarrow [7, 6, 3, 2, \cancel{5}]$

55 45 60 15
 $\longleftarrow$
 175